# Code Python, Consult AI

Python Fundamentals for the AI Era

Michael Borck

**Code Python, Consult AI: Python Fundamentals for the AI Era**

Copyright © 2026 Michael Borck.

Book content: Creative Commons Attribution 4.0 International (CC BY 4.0 International).
https://creativecommons.org/licenses/by/4.0/

Original code examples and accompanying software: MIT License.
https://opensource.org/licenses/MIT

These grants cover material the author has authority to license. Separately credited third-party material retains its stated terms.

Published by Michael Borck
Perth, Western Australia

**ISBN:** 979-8-254-65364-6

First edition, 2026.

**AI disclosure:** This book was written using the methodology it describes. AI tools were used as thinking partners throughout the drafting, iterating, and refining process. The author reviewed, challenged, and took responsibility for every sentence and line of code.

**Companion website:** https://michael-borck.github.io/code-python-consult-ai
**Source:** https://github.com/michael-borck/code-python-consult-ai

# Table of contents


Preface . . . . . 1
1 After Vibe Coding . . . . . 7
2 Getting Started with Python . . . . . 13
3 Values . . . . . 19
4 Variables . . . . . 25
5 Output . . . . . 31
6 Input . . . . . 37
7 Operators . . . . . 43
8 Using Functions . . . . . 49
9 Creating Functions . . . . . 55
10 Making Decisions . . . . . 63
11 Lists . . . . . 69
12 Loops . . . . . 75
13 Strings . . . . . 83
14 Dictionaries . . . . . 89
15 Files . . . . . 97
16 Errors and Exceptions . . . . . 103
17 Debugging . . . . . 109
18 Testing . . . . . 115
19 Modules and Packages . . . . . 121
20 Objects . . . . . 127
Acknowledgments . . . . . 135
About the Author . . . . . 137

**Appendices** **139**
A Setting Up Python . . . . . 139

# Preface

## Why This Book Exists

You have been using AI to write code. Maybe it started with a simple request, "build me a web scraper" or "make a script that renames these files", and it worked. Then you got more ambitious. The code got longer. And at some point, you hit the wall.

The wall is that moment when AI gives you code and you cannot tell whether it is correct. You paste an error back in and get a different error. You ask for a fix and the whole approach changes. You cannot tell which variables matter, what the logic is doing, or why it broke.

This book exists because of the wall.

But the wall is not the only reason to learn Python. AI keeps getting smarter, and the wall keeps moving further out. What does not move is curiosity. The desire to understand what is happening under the hood, not because you have to, but because you want to. Because knowing how code works makes you better at directing AI, better at evaluating its output, and better at building things that actually do what you need.

Whether you hit the wall and need to get past it, or you are simply curious about how the code works, this book gives you that understanding.

## How This Book Works

This is not a traditional Python textbook. You will not read pages of explanations and then try exercises at the end. Instead, you will learn through conversation with AI.

Every chapter follows the same pattern:

**The Wall**. A specific vibe-coding failure you will recognise. AI produced code that broke or confused you, and you could not diagnose why. This is the concept you are about to learn.

**Thinking Session**. You open a conversation with your AI assistant and explore the concept. Guided prompts help you ask the right questions, push back on oversimplified answers, and build genuine understanding. No code writing yet. Just thinking.

**The Gap**. A brief pause where the author connects what you learned to what you are about to build.

**Building Session**. You open a fresh AI conversation with clear context and direct AI to build something specific. The chatbot project evolves with each chapter, and you understand every line of what AI produces because you explored the concept first.

**Quick Reference**. A minimal syntax card for future reference.

The two-chat structure used in every chapter comes from the *Conversation, Not Delegation* framework and is detailed in *Converse Python, Partner AI*. You do not need to read either to use this book. The method is built into every chapter. Understanding compounds. Each concept makes the next one easier. Vibe coding does not compound. Each session starts from zero.

## Who This Book Is For

- People who have used AI to write code and want to understand what it produces
- Professionals who need enough Python to evaluate, modify, and direct AI-generated code
- Anyone curious about how programming works under the hood
- Readers of *Think Python, Direct AI* ready for deeper language knowledge

If you worked through *Think Python, Direct AI*, you already used variables, functions, lists, and dictionaries to build projects. This book makes that knowledge systematic. You understand how Python works, not just how to use it.

If you are starting fresh, the book stands on its own. Each chapter's Thinking Session builds understanding from the ground up through AI conversation.

## What This Book Is Not

This is not a computer science textbook. It does not cover algorithms, data structures theory, or computational thinking. That is what *Think Python, Direct AI* covers.

It is not a professional practices book. Project structure, testing frameworks, deployment, and CI/CD are covered in *Ship Python, Orchestrate AI*.

It is not a reference manual. It covers the fundamentals you need to read and direct AI-generated code. When you need more, your AI assistant and the official documentation will fill the gaps, and you will be equipped to evaluate what they give you.

## The Chatbot Project

Across every chapter, you build a chatbot that grows from a 10-line echo script to a full-featured conversational program. By the end:

- It remembers your name and tracks conversations
- It matches keywords and categorises messages
- It picks responses from configurable dictionaries
- It saves conversation history to files
- It handles errors gracefully
- It has a test suite
- It is structured as a proper Python package with classes

You build this through AI conversation, not by copying code from a textbook. Each Building Session adds one feature, and you understand every line because you explored the underlying concept in the Thinking Session first.

## Conventions Used in This Book

AI prompts appear in callout boxes:

> **Thinking Session Prompt**
>
> What is the difference between a list and a dictionary in Python? When should I use each one?

Code examples appear with syntax highlighting:

```
name = input("What is your name? ")
print(f"Hello, {name}!")
```

Author guidance appears in tip boxes:

> **What to Look For**
>
> Your AI should explain that lists are ordered by position and dictionaries are ordered by key.

One note on the Building Session listings: from version to version they sometimes trim unrelated earlier features for space. When a listing shows only part of your chatbot, keep the features you have already built; each chapter tells you what to add, never what to remove.

## How This Book Was Written

This book was written through human-AI collaboration using the methodology it teaches. The structure, pedagogy, and editorial decisions were made by the author. Claude (Anthropic) assisted with drafting and refining. The process demonstrates the book's core message: AI is most useful when you understand what you are asking it to do.

## Ways to Engage with This Book

- **Read it online.** The full book is freely available at the companion website, with dark mode, search, and navigation.
- **Read it on paper or e-reader.** Available as a paperback and ebook through Amazon KDP.
- **Converse with it.** The online edition includes a chatbot grounded in the book's content.
- **Feed it to your own AI.** The `llm.txt` file provides a clean text version of the entire book, ready to paste into ChatGPT, Claude, or any AI tool.
- **Run the code.** Project code is available on GitHub (https://github.com/michael-borck/code-python-consult-ai). DeepWiki (https://deepwiki.com/michael-borck/code-python-consult-ai) provides an AI-navigable view of the repository.
- **Browse all books.** This book is part of a series. See all titles at books.borck.education (https://books.borck.education).

The online version is always the most current.

### Source Code & Feedback

All code examples and supplementary materials are available at: https://github.com/michael-borck/code-python-consult-ai

Found an error? Have a suggestion?

- Open an issue: https://github.com/michael-borck/code-python-consult-ai/issues
- Email: michael@borck.me

## The Series

This book is part of a series designed to help you master modern software development in the AI era.

The series has a trunk: *Conversation, Not Delegation* for method and *The Human Edge* for professional judgement. This book is the Python fundamentals branch.

Two ways in:

| Start with | Continue with | Because |
|---|---|---|
| *Conversation, Not Delegation* | *Build Web, Guide AI*, or *Converse Python, Partner AI* | You want the ideas before (or without) code |
| *Think Python, Direct AI* | *Code Python, Consult AI* (this book), then *Ship Python, Orchestrate AI* | You want to code from your first page |

*Converse Python, Partner AI* can be read alongside either path; its methodology enhances every one.

- **Conversation, Not Delegation**: the general methodology for working with AI across any discipline. No code required; the broadest entry point.
- **Think Python, Direct AI**: computational thinking for absolute beginners. The entry point if you want to code from the start.
- **Converse Python, Partner AI**: the intentional prompting methodology applied to software development. A companion you can read alongside any Python book.
- **Code Python, Consult AI** (this book): focused Python fundamentals through AI conversation.
- **Ship Python, Orchestrate AI**: professional Python development practices and tooling.
- **The Absolute Minimum You Must Know**: the free, always-current web companion at minimum.borck.education (https://minimum.borck.education), quick primers on every topic this series covers in depth.
- **Build Web, Guide AI**: web development with AI as your development partner.

All titles are available at books.borck.education (https://books.borck.education).

# Chapter 1

# After Vibe Coding

## 1.1 The Honeymoon

You discovered AI-assisted coding. You typed a few sentences into ChatGPT or Claude or Gemini, and it gave you working Python code. You ran it. It worked. You felt powerful.

You built a web scraper. A data visualisation. Maybe even a small app. You did not need to understand the code. You just needed to describe what you wanted. When something broke, you pasted the error back into AI and asked it to fix it. Sometimes that worked too.

This is vibe coding. Code by feel, prompt until something works, do not worry about understanding the details. For simple, self-contained scripts, it is genuinely effective.

## 1.2 The Wall

Then you hit the wall.

The AI gave you code with ten variables and you could not tell which ones mattered. It used a dictionary when a list would have worked. It wrapped everything in try/except and swallowed the actual error. You asked it to fix a bug and it introduced two more. You pasted the new error in and got a completely different approach that also did not work.

The problem was not AI. The problem was that you could not read the code it produced. You could not tell correct from plausible. You could not describe what was wrong precisely enough for AI to actually help.

This is the wall every vibe coder hits. Not on day one, on day thirty, or day sixty, when the code gets complex enough that "just make it work" stops working.

## 1.3 What This Book Does

This book gives you enough Python to stay on the right side of the wall. Not to become a software engineer, to become someone who can read AI-generated code, spot when it is wrong, and direct AI precisely enough to get what you actually need. Put differently: AI is very good at producing a first draft. Your job is shifting from writing first drafts to evaluating and editing them: spotting the draft that is confidently wrong, asking for the change that actually fixes it, and knowing the difference. That is a higher-order skill than writing the code yourself, and it is the skill this book builds.

You will learn through conversation with AI, not by reading textbook explanations. Every chapter follows the same pattern:

### 1.3.1 The Two-Chat Method

**Chat 1: Thinking Session.** Open a conversation with your AI assistant. Explore the concept. Ask questions. Challenge the AI's explanations. Push back when something does not make sense. Build understanding before writing a single line of code.

**The Gap.** Step back. Reflect on what you learned. Decide what matters for what you are building.

**Chat 2: Building Session.** Open a fresh conversation. This time, you have context. You write a specific, informed prompt. The AI produces better code because your input is better. And you can read what it produces because you understand the concept.

This is the difference between vibe coding and intentional prompting. Vibe coding does not compound. Each session starts from zero. Understanding compounds. Each concept makes the next one easier.

In current terms, this is *context engineering*. When you run a Thinking Session and then carry only your decisions into a fresh Building Session, you are doing the one thing that makes an AI assistant, or an autonomous agent, genuinely useful: curating the context it works from, and refusing to let it inherit a messy, half-explored conversation. The same habit scales from a two-chat workflow to directing a coding agent that edits your files: the quality of what it produces is bounded by the quality of the brief you hand it. Get the thinking right first, and the building, by you or by an agent, gets dramatically easier.

## 1.4 Your First Thinking Session

Open your AI assistant. Paste this prompt:

**Thinking Session Prompt**

I have been using you to write Python code for me, but I do not really understand what is happening in the code you produce. I want to learn the fundamentals so I can stay in control.
What are the five most important Python concepts I should understand first if my goal is to be able to read and evaluate AI-generated code? For each one, give me a one-sentence explanation and a one-line code example.

Read the response carefully. Your AI will likely mention variables, data types, functions, control flow, and something about data structures. These are the chapters in this book.

Now push back:

**Thinking Session Prompt**

You listed five concepts. But when I look at AI-generated Python code, I also see things like try/except, import statements, and classes. Where do those fit? Are they more or less important than the five you listed?

**What to Look For**

Your AI should explain that the first five concepts are foundational, everything else builds on them. If it just lists more concepts without prioritising, ask it to rank them by how often they appear in typical AI-generated code.

This is what a Thinking Session feels like: exploring, questioning, building a mental map. You are not trying to write code yet. You are trying to understand the territory.

## 1.5 Your First Building Session

Now open a fresh conversation. You are going to build the project that evolves across every chapter: a terminal chatbot.

**Building Session Prompt**

Write me a minimal Python chatbot that runs in a terminal. It should:

- Print a greeting with the bot's name
- Accept user messages in a loop
- Echo the message back with a simple response

- Exit when the user types “quit”

Keep it under 15 lines. Use only basic Python. No imports, no classes, no functions beyond print() and input().

Your AI will produce something like this:

```
bot_name = "PyBot"
print(f"Hello! I'm {bot_name}. Type 'quit' to
exit.")

while True:
    user_input = input("You: ")
    if user_input.lower() == "quit":
        print(f"{bot_name}: Goodbye!")
        break
    print(f"{bot_name}: You said '{user_input}'")
```

**What to Notice**

Even in 10 lines, you can see several concepts: a variable (`bot_name`), a string with formatting (`f"..."`), a loop (`while True`), a decision (`if`), and a way to stop (`break`). You do not need to understand all of these yet. Each chapter covers one. By the end of the book, you will understand every line.

Run this code. Talk to it. It is not smart. It just echoes what you say. But it is the skeleton that you will build on. By chapter 20, it will have memory, pattern matching, file persistence, error handling, and a proper class structure.

## 1.6 What You Need

- **Python installed.** If you do not have it yet, see the appendix on Setting Up Python.
- **An AI assistant.** Any will do. ChatGPT, Claude, Gemini, or whatever you prefer. The prompts in this book work with all of them.
- **A willingness to read code.** Not just run it, read it, question it, understand it. That is the skill this book teaches.

## 1.7 How This Book Is Different

There are hundreds of Python tutorials. This one does not explain variables the way a textbook does. Instead, it shows you how to explore variables through

conversation with AI, then apply what you learned by directing AI to build something specific.

Every chapter follows the same two-chat structure. By the end, you will not just know Python. You will know how to learn anything through AI conversation, and how to direct AI to build what you actually need.

That is the skill that outlasts any specific tool or framework.

It is also the skill that keeps you valuable as the tools improve. When AI can write competent code for anyone, the person who can read, evaluate, and steer it is the one adding the variation competent sameness cannot, and that is the one thing that does not commoditise. The full argument is in *Conversation, Not Delegation*.

# Chapter 2

# Getting Started with Python

## 2.1 The Wall

AI gave you a Python script. You copied it into a file, double-clicked it, and nothing happened. Or you pasted it into a terminal and got `SyntaxError: invalid syntax`. Or it ran but the output disappeared instantly.

You asked AI how to run Python code and got a wall of text about interpreters, virtual environments, IDEs, and Jupyter Notebooks. You just wanted to run the file.

This chapter fixes that. You will learn how Python code runs, how to read the structure AI gives you, and why indentation and naming actually matter when you are trying to modify what AI produces.

## 2.2 Thinking Session

### 2.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> I have a Python file that AI generated for me. When I try to run it, I get errors or nothing happens. Can you explain the different ways to run Python code? I want to understand: what is the difference between running code in a Jupyter Notebook, in a terminal, and in a script file? When would I use each one?

Your AI should explain three main approaches: the interactive interpreter (type code line by line), script files (save and run a `.py` file), and Jupyter Notebooks

(cells that mix code and output). If it jumps straight to IDE setup, pull it back to the basics first.

### 2.2.2 Go Deeper

**Thinking Session Prompt**

When I look at AI-generated Python code, I notice it uses indentation to structure things, unlike other languages that use curly braces. Why does Python do this? What happens if I get the indentation wrong? Show me an example of code that breaks because of indentation.

**What to Look For**

Your AI should explain that Python uses indentation to define code blocks. It is not decorative, it is structural. A misplaced space causes an `IndentationError`. This matters because when you copy AI-generated code and reformat it, you can accidentally break the structure.

**Thinking Session Prompt**

AI-generated code often has comments like `# This is a comment` and names like `user_name` or `MAX_RETRIES`. Are there rules for how to name things in Python? What conventions do Python programmers follow, and why should I care when I am mostly reading AI-generated code?

The naming conventions matter because they carry meaning. When you see `MAX_RETRIES`, the uppercase tells you it is a constant. When you see `user_name`, the snake_case tells you it is a variable. When you see `UserAccount`, the PascalCase tells you it is a class. If you do not know these conventions, AI-generated code looks like a random collection of names.

### 2.2.3 Challenge It

**Thinking Session Prompt**

Here is some Python code with several problems. Can you identify all of them?

```
if x > 10
    Print("x is big")
  print("done")
```

```
message = "Hello, world!'
```

**What to Look For**

Four errors: missing colon after `if x > 10`, capital P in `Print` (Python is case-sensitive), inconsistent indentation on `print("done")`, and mismatched quotes on the string. These are exactly the errors you will see when you edit AI-generated code carelessly.

### 2.2.4 What You Should Have Learned

After your Thinking Session, you should be able to:

- Run Python code three ways: interpreter, script, notebook
- Explain why indentation matters in Python (it defines structure, not style)
- Read Python naming conventions: snake_case, PascalCase, UPPER_CASE
- Spot common syntax errors: missing colons, wrong case, bad indentation, mismatched quotes

If any of these are unclear, continue the conversation with your AI before moving on.

## 2.3 The Gap

You now know how to run Python and how to read the structure of Python code. When AI generates code with indented blocks, comments, and named variables, you understand what those structural elements mean, even before you understand what the code does.

In the Building Session, you will set up your Python environment and add proper structure to the chatbot from Chapter 1: comments, named constants, and clear formatting.

## 2.4 Building Session

### 2.4.1 The Spec

Take the chatbot skeleton from Chapter 1 and add proper Python structure:

- Add a descriptive comment at the top explaining what the code does
- Use named constants for the bot name and version (UPPER_CASE)
- Add inline comments on any non-obvious lines
- Make sure indentation is consistent (4 spaces)

### 2.4.2 Prompt It

**Building Session Prompt**

Here is my chatbot from Chapter 1:

```
bot_name = "PyBot"
print(f"Hello! I'm {bot_name}. Type 'quit' to
exit.")

while True:
    user_input = input("You: ")
    if user_input.lower() == "quit":
        print(f"{bot_name}: Goodbye!")
        break
    print(f"{bot_name}: You said '{user_input}'")
```

Improve it by adding: - A module docstring at the top describing the program - BOT_NAME and VERSION as constants (uppercase) - Inline comments explaining the while loop and the break - Keep it simple. No functions, no imports

### 2.4.3 Read the Code

Your AI will produce something like this:

```
"""
PyBot: A simple terminal chatbot.
Greets the user and echoes their messages.
Type 'quit' to exit.
"""

# Configuration
BOT_NAME = "PyBot"
VERSION = "0.2"

print(f"Hello! I'm {BOT_NAME} v{VERSION}.")
print("Type 'quit' to exit.")

# Main conversation loop (runs until user quits)
while True:
    user_input = input("You: ")

    if user_input.lower() == "quit":
```

```
        print(f"{BOT_NAME}: Goodbye!")
        break  # Exit the loop

    print(f"{BOT_NAME}: You said '{user_input}'")
```

> **What to Notice**
>
> The docstring at the top (triple quotes) describes the program. `BOT_NAME` and `VERSION` are uppercase. The naming convention signals these are constants that should not change. The comments explain *why* (the loop runs until quit) not *what* (print prints). The `break` comment is useful because `break` is a keyword you might not recognise yet.

### 2.4.4 Stretch It

> **Building Session Prompt**
>
> Add a comment block after the constants that lists what the chatbot can do in this version. Format it as a multi-line comment. Also add a `# TODO:` comment listing two features we will add in future chapters.

Read the result. `# TODO:` comments are a convention for marking planned work. You will see these frequently in AI-generated code and in professional projects.

## 2.5 Your Chatbot So Far

After this chapter, your chatbot can:

- Print a greeting with the bot's name and version
- Accept user messages in a loop
- Echo messages back
- Exit when the user types "quit"
- **Has proper structure: docstring, constants, comments**

## 2.6 Quick Reference

```
# Comment styles
# Single-line comment
x = 5  # Inline comment
"""Docstring for documentation."""
```

```
# Naming conventions
user_name = "alice"     # snake_case: variables
MAX_RETRIES = 3         # UPPER_CASE: constants
class UserAccount:      # PascalCase: classes
    pass

# Indentation: 4 spaces per level
if True:
    print("indented")

# Running Python
# Terminal:  python3 my_script.py
# Notebook:  Shift+Enter to run cell
```

# Chapter 3

# Values

## 3.1 The Wall

You asked AI to build a temperature converter. It produced code that multiplied a string by a number and crashed with `TypeError: can't multiply sequence by non-int of type 'float'`. You had no idea what a "sequence" was or why multiplying was a problem. The formula looked right.

The issue was data types. AI treated the input as text when it needed to be a number. If you do not know the difference between `"72"` and `72`, you cannot spot this in AI-generated code, and AI generates this bug constantly.

This chapter fixes that.

## 3.2 Thinking Session

### 3.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> In Python, what is the difference between the number 42 and the text "42"? Why does Python treat them differently, and what goes wrong when you mix them up? Give me examples of errors that happen when types get confused.

Your AI should explain that `42` is an integer and `"42"` is a string, and that Python does not automatically convert between them. If your AI says "Python is dynamically typed so it handles this for you," push back, dynamic typing means variables can change type, not that types do not matter.

### 3.2.2 Go Deeper

**Thinking Session Prompt**

What are the main data types in Python? I do not need all of them. Just the ones I will see most often in AI-generated code. For each one, show me what it looks like and one thing that catches people off guard about it.

**What to Look For**

Your AI should cover: `int` (integers), `float` (decimals), `str` (text), `bool` (True/False), and `None`. The gotchas matter: float division can produce unexpected precision (`0.1 + 0.2` is not `0.3`), booleans are actually integers (`True + True` is 2), and `None` is not the same as `0` or `""`.

**Thinking Session Prompt**

How do I convert between types in Python? If AI gives me a string from user input and I need a number, how do I convert it? What happens if the conversion fails?

### 3.2.3 Challenge It

**Thinking Session Prompt**

What will each of these produce, and why?

```
"5" + "3"
5 + 3
"5" + 3
int("5") + 3
float("hello")
```

**What to Look For**

`"5" + "3"` gives `"53"` (string concatenation). `5 + 3` gives `8`. `"5" + 3` crashes with TypeError. `int("5") + 3` gives `8`. `float("hello")` crashes with ValueError. If you can explain all five, you understand values.

### 3.2.4 What You Should Have Learned

- Python has distinct types: int, float, str, bool, None
- `"42"` and `42` are fundamentally different

- Type conversion is explicit: `int()`, `float()`, `str()`
- Conversion can fail: `int("hello")` crashes
- The `type()` function tells you what you are working with

## 3.3 The Gap

When AI generates code that processes user input, it almost always needs type conversion. The input comes in as a string, but the code needs a number. Now you can spot that moment in the code and verify AI handled it correctly.

In the Building Session, you will make your chatbot respond differently to numbers versus text, the first step toward intelligent responses.

## 3.4 Building Session

### 3.4.1 The Spec

Make your chatbot recognise different types of input:

- If the user types a number, respond with a fun fact about it (even/odd, big/small)
- If the user types text, echo it as before
- Show the type of the input in debug-style output

### 3.4.2 Prompt It

Building Session Prompt

Here is my chatbot so far:

```
"""PyBot v0.2: Terminal chatbot with structure."""
BOT_NAME = "PyBot"
VERSION = "0.2"

print(f"Hello! I'm {BOT_NAME} v{VERSION}.")
print("Type 'quit' to exit.")

while True:
    user_input = input("You: ")
    if user_input.lower() == "quit":
        print(f"{BOT_NAME}: Goodbye!")
        break
    print(f"{BOT_NAME}: You said '{user_input}'")
```

> Update to v0.3. Add logic that: - Tries to convert the input to a number using int() - If it succeeds, responds with whether the number is even or odd - If it fails, treats the input as text and echoes it - Prints the type of the input in parentheses after the response - Use a try/except for the conversion (just the basic pattern)
> Keep it simple. No functions yet, keep the while loop structure.

### 3.4.3 Read the Code

Your AI will produce something like this:

```
"""PyBot v0.3: Recognises numbers vs text."""
BOT_NAME = "PyBot"
VERSION = "0.3"

print(f"Hello! I'm {BOT_NAME} v{VERSION}.")
print("Type 'quit' to exit.")

while True:
    user_input = input("You: ")

    if user_input.lower() == "quit":
        print(f"{BOT_NAME}: Goodbye!")
        break

    # Try to treat input as a number
    try:
        number = int(user_input)
        if number % 2 == 0:
            response = f"{number} is even"
        else:
            response = f"{number} is odd"
        input_type = type(number).__name__
    except ValueError:
        response = f"You said '{user_input}'"
        input_type = type(user_input).__name__

    print(f"{BOT_NAME}: {response}
    ({input_type})")
```

> **What to Notice**
>
> Line 16 uses `int(user_input)`, explicit type conversion. If it fails, the `except ValueError` catches it and the code falls through to the string handling. `type(number).__name__` gives the type as a readable string. The `%` operator checks even/odd. You will learn more about operators in Chapter 7.

### 3.4.4 Stretch It

> **Building Session Prompt**
>
> Also handle float input. If the user types "3.14", recognise it as a decimal number and respond with "3.14 rounded down is 3". Try float() first, then int(), then fall back to string.

## 3.5 Your Chatbot So Far

- Ch 1: Basic loop with greeting and quit
- Ch 2: Docstring, constants, comments
- **Ch 3: Recognises numbers vs text, shows input type**

## 3.6 Quick Reference

```
# Types
x = 42          # int
x = 3.14        # float
x = "hello"     # str
x = True        # bool
x = None        # NoneType

# Check type
type(x)             # <class 'int'>
type(x).__name__    # 'int'
isinstance(x, int)  # True

# Convert
int("42")       # 42
float("3.14")   # 3.14
str(42)         # "42"
```

```
bool(0)          # False
bool("")         # False
```

# Chapter 4

# Variables

## 4.1 The Wall

You asked AI to "make a chatbot that remembers the user's name." It gave you 40 lines of code with `user_name`, `bot_name`, `message_count`, `last_response`, `_internal_state`, and `MAX_RETRIES`. It worked, until you tried to change something and everything broke. You renamed a variable in one place but not another. You changed a value that turned out to be used in three different places.

You could not tell which variables mattered, which were temporary, and which were constants you should not touch. Without understanding how variables work, AI-generated code is a minefield of names you cannot safely modify.

This chapter fixes that.

> **Coming from Think Python?**
>
> If you worked through *Think Python, Direct AI*, you already used variables in projects like the Temperature Converter and Quiz Game. This chapter makes that knowledge systematic, explaining the rules and patterns behind what you practised.

## 4.2 Thinking Session

### 4.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> Explain Python variables to me, but do not use the "box that holds a value" metaphor. I want to understand what actually happens when I write `x = 5`. Also explain why Python lets me write `x = 5` and then `x = "hello"` on the next line, is that a feature or a problem?

Your AI should talk about name binding. `x = 5` binds the name `x` to the integer object `5`. The "box" metaphor is misleading because it implies the variable contains the value. In Python, variables are labels pointing to objects. If your AI uses the box metaphor anyway, push back.

### 4.2.2 Go Deeper

> **Thinking Session Prompt**
>
> Show me three examples where choosing bad variable names would make AI-generated code confusing to modify. Then show the same code with good names. What naming conventions does Python use?

> **What to Look For**
>
> The pattern: `snake_case` for variables and functions, `UPPER_CASE` for constants, `PascalCase` for classes. These are not just style. They carry meaning. When you see `MAX_RETRIES` in AI-generated code, the uppercase tells you it is a constant that should not change.

> **Thinking Session Prompt**
>
> What is the difference between `x = 5` and `x == 5`? When I am reading AI-generated code, how do I know which one I am looking at? And what about `x += 1`, what does that do?

### 4.2.3 Challenge It

**Thinking Session Prompt**

What is wrong with this code, and why would AI likely generate it?

```
name = "Alice"
def greet():
    print(name)
    name = name + "!"
```

**What to Look For**

This is a scope trap. The `name = name + "!"` inside the function creates a local variable that shadows the global one. Python sees the assignment and treats `name` as local for the entire function, so `print(name)` fails with `UnboundLocalError`. AI generates this pattern regularly.

### 4.2.4 What You Should Have Learned

- Variables are names bound to objects, not boxes holding values
- Assignment (=) vs equality (==) vs augmented assignment (+=)
- Python naming conventions: snake_case, UPPER_CASE, PascalCase
- Dynamic typing means variables can change type, powerful but requires attention
- Scope exists and can cause subtle bugs

## 4.3 The Gap

You can now read AI-generated code and understand what the variable names mean, why they are structured the way they are, and what happens when values change. That is the difference between someone who copies AI output and someone who can safely modify it.

In the Building Session, you will add memory to your chatbot, and understand every variable AI creates.

## 4.4 Building Session

### 4.4.1 The Spec

Add state tracking to your chatbot:

- Store the user's name (ask for it at the start)
- Track how many messages have been exchanged
- Use named constants for bot configuration
- Reference the stored name in responses

### 4.4.2 Prompt It

Building Session Prompt

Here is my chatbot (v0.3). Update it to v0.4:

```
"""PyBot v0.3: Recognises numbers vs text."""
BOT_NAME = "PyBot"
VERSION = "0.3"
print(f"Hello! I'm {BOT_NAME} v{VERSION}.")
print("Type 'quit' to exit.")
while True:
    user_input = input("You: ")
    if user_input.lower() == "quit":
        print(f"{BOT_NAME}: Goodbye!")
        break
    try:
        number = int(user_input)
        response = f"{number} is {'even' if number
        % 2 == 0 else 'odd'}"
    except ValueError:
        response = f"You said '{user_input}'"
    print(f"{BOT_NAME}: {response}")
```

Add these features using variables: - Ask the user's name at the start, store in user_name - Add message_count that increments each turn - Include user's name and count in responses - Update VERSION to "0.4"
Keep it simple. No functions, no classes.

### 4.4.3 Read the Code

Your AI will produce something like this:

```
"""PyBot v0.4: Remembers your name, tracks
messages."""
BOT_NAME = "PyBot"
VERSION = "0.4"
```

```
print(f"Hello! I'm {BOT_NAME} v{VERSION}.")
user_name = input(f"{BOT_NAME}: What's your name?
")
print(f"{BOT_NAME}: Nice to meet you,
{user_name}!")
print("Type 'quit' to exit.")

message_count = 0

while True:
    user_input = input(f"{user_name}: ")

    if user_input.lower() == "quit":
        print(f"{BOT_NAME}: Goodbye,
        {user_name}!")
        print(f"We exchanged {message_count}
        messages.")
        break

    message_count += 1

    try:
        number = int(user_input)
        response = f"{number} is {'even' if number
        % 2 == 0 else 'odd'}"
    except ValueError:
        response = f"You said '{user_input}'"

    print(f"{BOT_NAME}: {response} (msg
    #{message_count})")
```

**What to Notice**

`user_name` stores the result of `input()`. It persists across the entire loop. `message_count` starts at `0` and uses `+= 1` to increment. Both `BOT_NAME` and `VERSION` are constants (uppercase) set once and never changed. The f-strings reference these variables by name, change the variable, change every response.

### 4.4.4 Stretch It

> **Building Session Prompt**
>
> Add a `favourite_topic` variable that starts as None. When the user mentions "python" or "coding", set it to that word. In future responses, if favourite_topic is set, occasionally mention it.

Notice how your AI handles the `None` initial value and the `is not None` check. This pattern appears constantly in AI-generated code.

## 4.5 Your Chatbot So Far

- Ch 1: Basic loop with greeting and quit
- Ch 2: Docstring, constants, comments
- Ch 3: Recognises numbers vs text
- **Ch 4: Remembers user name, tracks message count**

## 4.6 Quick Reference

```
# Assignment
x = 5
x = "hello"         # dynamic typing
x += 1               # augmented assignment
x, y = 1, 2          # multiple assignment
x, y = y, x          # swap

# Constants (convention)
BOT_NAME = "PyBot"
MAX_RETRIES = 3

# None
x = None
if x is not None:
    print(x)
```

# Chapter 5

# Output

## 5.1 The Wall

Your AI-generated program ran without errors but the output was a mess. Numbers had too many decimal places. Columns did not line up. There were no blank lines between sections and everything ran together. You asked AI to "make it look nicer" and got code full of `f"{value:>10.2f}"`, formatting syntax that looked like someone spilled punctuation on the keyboard.

You need to understand `print()` and string formatting, not because you will memorise every format code, but because AI uses them constantly and you need to know what you are looking at when you modify the output.

This chapter fixes that.

## 5.2 Thinking Session

### 5.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> How does Python's print() function work beyond the basics? I know `print("hello")` prints text, but AI-generated code uses things like `sep=`, `end=`, and `f""` strings. Explain what these do and when they matter.

Your AI should cover three things: `print()` parameters (`sep` changes what goes between items, `end` changes what goes at the end, default is newline), f-strings (formatted string literals that embed expressions in `{}`), and format specifications (the `:` syntax inside `{}`).

### 5.2.2 Go Deeper

**Thinking Session Prompt**

Show me how f-strings work in Python. I want to understand the syntax inside the curly braces, how do I control decimal places, alignment, padding, and width? Give me examples I would actually use, not theoretical ones.

**What to Look For**

Key patterns: `{value:.2f}` for 2 decimal places, `{text:<20}` for left-aligned in 20 chars, `{text:>20}` for right-aligned, `{text:^20}` for centred. These appear in AI-generated table output, reports, and formatted displays.

**Thinking Session Prompt**

What is the difference between print(), f-strings, .format(), and % formatting in Python? AI sometimes uses different ones in the same code. Which should I use and which are outdated?

### 5.2.3 Challenge It

**Thinking Session Prompt**

What does this code print, and why?

```
print("Name", "Age", "Score", sep=" | ")
print(f"{'Alice':.<15}{85:>5}")
print("Loading", end="")
print(".", end="")
print(".", end="")
print(" Done!")
```

**What to Look For**

Line 1 prints `Name | Age | Score` (sep changes the separator). Line 2 prints `Alice.....  85` (left-aligned with dot fill, right-aligned number). Lines 3-5 print `Loading.. Done!` on one line (end="" suppresses the newline). These are tricks AI uses to create formatted output.

### 5.2.4 What You Should Have Learned

- `print()` has `sep` and `end` parameters that change formatting
- f-strings are the modern way to embed values in text: `f"Hello {name}"`
- Format specs control alignment, width, and decimal places: `{value:.2f}`
- `end=""` prevents newlines, useful for progress indicators
- `.format()` and `%` still exist but f-strings are preferred

## 5.3 The Gap

You can now read AI-generated output code and understand what all the formatting syntax does. When AI produces a table with aligned columns or a progress bar, you know how to adjust the widths, change the alignment, or add decimal places.

In the Building Session, you will give your chatbot a proper welcome screen with formatted output.

## 5.4 Building Session

### 5.4.1 The Spec

Add formatted output to your chatbot:

- A welcome banner with the bot name centred in a bordered box
- Display the user's name and message count in aligned format
- Show a separator line between the welcome and the conversation

### 5.4.2 Prompt It

> **Building Session Prompt**
>
> Update my chatbot to v0.5. Add a formatted welcome screen when the bot starts:
> - A line of 50 equals signs
> - The bot name centred within 50 characters
> - "Your Python Assistant" centred below it
> - Another line of 50 equals signs
> - After asking the user's name, show a status line: "User: {name}" left-aligned, "Messages: 0" right-aligned, all within 50 characters
>
> Use f-strings with format specifications for the alignment. Keep the rest of the chatbot logic from v0.4.

### 5.4.3 Read the Code

Your AI will produce something like this:

```
"""PyBot v0.5: Formatted output."""
BOT_NAME = "PyBot"
VERSION = "0.5"
WIDTH = 50

print("=" * WIDTH)
print(f"{BOT_NAME:^{WIDTH}}")
print(f"{'Your Python Assistant':^{WIDTH}}")
print("=" * WIDTH)

user_name = input(f"{BOT_NAME}: What's your name?
")
message_count = 0

print(f"{'User: ' + user_name:<{WIDTH // 2}}"
      f"{'Messages: ' + str(message_count):>{WIDTH
      // 2}}")
print("-" * WIDTH)

while True:
    user_input = input(f"{user_name}: ")

    if user_input.lower() == "quit":
        print("-" * WIDTH)
        print(f"Goodbye, {user_name}! "
              f"({message_count} messages)")
        break

    message_count += 1

    try:
        number = int(user_input)
        response = f"{number} is {'even' if number
        % 2 == 0 else 'odd'}"
    except ValueError:
        response = f"You said '{user_input}'"

    print(f"{BOT_NAME}: {response}")
```

**What to Notice**

`{BOT_NAME:^{WIDTH}}` centres the text in a field of WIDTH characters. `{:>{WIDTH // 2}}` right-aligns in half the width. `"=" * WIDTH` repeats the character. A common pattern for separator lines. The `WIDTH` constant means you change one number to adjust all formatting.

### 5.4.4 Stretch It

**Building Session Prompt**

After the conversation ends, display a summary table showing the message count, the user's name, and the bot version, formatted with aligned columns and a border.

## 5.5 Your Chatbot So Far

- Ch 1: Basic loop with greeting and quit
- Ch 2: Docstring, constants, comments
- Ch 3: Recognises numbers vs text
- Ch 4: Remembers user name, tracks messages
- **Ch 5: Formatted welcome screen, aligned output**

## 5.6 Quick Reference

```
# print() parameters
print("a", "b", "c", sep=", ")    # a, b, c
print("Loading", end="")           # no newline

# f-strings
name = "Alice"
print(f"Hello {name}")             # Hello Alice
print(f"{3.14159:.2f}")            # 3.14

# Alignment (within width of 20)
print(f"{'left':<20}")             # left-aligned
print(f"{'right':>20}")            # right-aligned
print(f"{'centre':^20}")           # centred
print(f"{'dots':.<20}")            # fill with
dots
```

```
# Repetition
print("=" * 50)                    # 50 equals
signs
```

# Chapter 6

# Input

## 6.1 The Wall

AI built you a program that asked the user for their age. They typed "twenty-five" and it crashed. You asked AI to fix it. It added a try/except that silently ignored the error. Now the program did not crash, but it also did not work. The user typed invalid input and the program carried on as if nothing happened.

AI's default approach to user input is either "assume it is perfect" or "catch everything and ignore it." Neither is acceptable. You need to understand how `input()` works and how to validate what comes back.

This chapter fixes that.

## 6.2 Thinking Session

### 6.2.1 Getting Oriented

> **i** Thinking Session Prompt
>
> How does Python's input() function work? I know it reads from the keyboard, but what type does it return? And what happens if the user types nothing and just presses Enter?

Your AI should emphasise that `input()` always returns a string, even if the user types a number. An empty Enter returns an empty string `""`, not None. This is the source of most input-related bugs in AI-generated code.

### 6.2.2 Go Deeper

**Thinking Session Prompt**

What are good patterns for validating user input in Python? I want to handle these cases: - User types nothing (empty input) - User types the wrong type (text when I need a number) - User types something outside the expected range
Show me a robust pattern I can reuse, not just a simple try/except that ignores errors.

**What to Look For**

The robust pattern is a validation loop: keep asking until input is valid. This typically uses `while True` with a `break` when input passes validation. Your AI should show this pattern rather than a single try/except.

**Thinking Session Prompt**

When I look at AI-generated code, I often see `input().strip().lower()` chained together. What does each part do, and why is this pattern so common?

### 6.2.3 Challenge It

**Thinking Session Prompt**

What is wrong with this input validation code?

```
age = input("Enter your age: ")
if age > 0 and age < 150:
    print(f"You are {age} years old")
```

**What to Look For**

`age` is a string (from `input()`), so `age > 0` compares a string to an integer. This raises a TypeError in Python 3. The fix is `age = int(input("Enter your age: "))` or validating and converting separately. AI generates this exact bug frequently.

### 6.2.4 What You Should Have Learned

- `input()` always returns a string, even for numbers
- Empty input returns `""`, not None
- `.strip()` removes whitespace, `.lower()` normalises case
- Validation loops (while True + break) are better than silent try/except
- Chain methods: `input().strip().lower()` is idiomatic Python

## 6.3 The Gap

You now understand the most common source of bugs in AI-generated interactive programs: input that is not validated or not converted. When you see AI code that uses `input()`, you can immediately check whether it handles empty input, wrong types, and out-of-range values.

In the Building Session, you will make your chatbot handle bad input gracefully instead of crashing or ignoring errors.

## 6.4 Building Session

### 6.4.1 The Spec

Add input validation to your chatbot:

- Handle empty input (user just presses Enter)
- Clean up input with strip() and lower() for comparisons
- Add a “help” command that lists available commands
- Validate any numeric input with a proper loop

### 6.4.2 Prompt It

> **Building Session Prompt**
>
> Update my chatbot to v0.6. Add input validation:
> - If the user enters empty input (just presses Enter), prompt them again with a helpful message instead of treating it as a message
> - Use .strip() on all input to handle accidental spaces
> - Use .lower() for command matching (quit, help)
> - Add a “help” command that lists: quit, help
> - When checking for numbers, if conversion fails, respond with a message about the input being text, do not silently ignore it
>
> Keep the existing features from v0.5 (welcome screen, name, message count, number recognition).

### 6.4.3 Read the Code

Your AI will produce something like this:

```
"""PyBot v0.6: Input validation."""
BOT_NAME = "PyBot"
VERSION = "0.6"
WIDTH = 50
COMMANDS = ["quit", "help"]

print("=" * WIDTH)
print(f"{BOT_NAME:^{WIDTH}}")
print("=" * WIDTH)

user_name = input(f"{BOT_NAME}: What's your name?
").strip()
while not user_name:
    user_name = input(f"{BOT_NAME}: Please enter
    your name: ").strip()

print(f"{BOT_NAME}: Nice to meet you,
{user_name}!")
message_count = 0

while True:
    user_input = input(f"{user_name}: ").strip()

    if not user_input:
        print(f"{BOT_NAME}: Say something! Type
        'help' for commands.")
        continue

    command = user_input.lower()

    if command == "quit":
        print(f"{BOT_NAME}: Goodbye,
        {user_name}!")
        break
    elif command == "help":
        print(f"{BOT_NAME}: Commands: {',
        '.join(COMMANDS)}")
        continue
```

```
    message_count += 1

    try:
        number = int(user_input)
        response = f"{number} is {'even' if number
        % 2 == 0 else 'odd'}"
    except ValueError:
        response = f"Interesting. '{user_input}'"

    print(f"{BOT_NAME}: {response} (msg
    #{message_count})")
```

> **What to Notice**
>
> The name input uses a `while not user_name` loop, keeps asking until something is entered. `.strip()` is called on every `input()` to remove accidental spaces. `command = user_input.lower()` creates a normalised version for comparison without changing the display. `continue` skips to the next loop iteration for empty input and help commands.

### 6.4.4 Stretch It

> **Building Session Prompt**
>
> Add an "age" command that asks the user for their age. Use a validation loop that keeps asking until they enter a valid number between 1 and 150.

## 6.5 Your Chatbot So Far

- Ch 1: Basic loop with greeting and quit
- Ch 2: Docstring, constants, comments
- Ch 3: Recognises numbers vs text
- Ch 4: Remembers user name, tracks messages
- Ch 5: Formatted welcome screen
- **Ch 6: Input validation, help command, strip/lower**

## 6.6 Quick Reference

```
# input() always returns a string
```

```
name = input("Name: ")            # str
age = int(input("Age: "))         # convert to int

# Clean input
text = input().strip()            # remove
whitespace
text = input().strip().lower()    # normalise

# Validation loop
while True:
    value = input("Enter a number: ").strip()
    try:
        number = int(value)
        break                     # valid, exit
        loop
    except ValueError:
        print("That's not a number. Try again.")

# Empty check
if not user_input:                # empty string is
falsy
    print("You didn't type anything")
```

# Chapter 7

# Operators

## 7.1 The Wall

AI gave you a calculation that used `//` and `%` and you thought `//` was a comment. It was not. It was integer division. The result was wrong because you did not know what the operators did, so you could not tell whether AI had used the right ones.

In another case, AI wrote `if x and y > 10:` and you assumed it checked whether both x and y were greater than 10. It did not. It checked whether x was truthy and y was greater than 10. Subtle, silent, wrong.

This chapter fixes that.

## 7.2 Thinking Session

### 7.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> What operators does Python have beyond the basic +, -, *, /? I keep seeing //, %, **, and, or, not, ==, !=, is, in, can you organise them into categories and explain when each one is used? Focus on the ones that AI uses most often in generated code.

Your AI should group them: arithmetic (`+`, `-`, `*`, `/`, `//`, `%`, `**`), comparison (`==`, `!=`, `<`, `>`, `<=`, `>=`), logical (`and`, `or`, `not`), membership (`in`, `not in`), and identity (`is`, `is not`). If it lists bitwise operators, you can skip those for now.

### 7.2.2 Go Deeper

**Thinking Session Prompt**

What is the difference between `/` and `//` in Python? And between `==` and `is`? These confuse me when I see them in AI-generated code. Give me examples where using the wrong one causes bugs.

**What to Look For**

`/` gives a float (`7 / 2` is `3.5`), `//` gives an integer (`7 // 2` is `3`). `==` checks value equality, `is` checks identity (same object in memory). The classic trap: `x == None` works but `x is None` is correct. AI usually gets this right, but not always.

**Thinking Session Prompt**

How does Python evaluate compound conditions? What does `if x and y > 10` actually check? And what about `if x or y`? I want to understand operator precedence in conditions because AI writes these frequently.

### 7.2.3 Challenge It

**Thinking Session Prompt**

What does each of these evaluate to, and why?

```
10 / 3
10 // 3
10 % 3
2 ** 10
"hello" * 3
"py" in "python"
not ""
True + True
```

**What to Look For**

`10 / 3` is `3.333...`, `10 // 3` is `3`, `10 % 3` is `1` (remainder). `2 ** 10` is `1024` (exponentiation). `"hello" * 3` is `"hellohellohello"` (string repetition). `"py" in "python"` is `True` (substring check). `not ""` is `True` (empty string

is falsy). `True + True` is `2` (booleans are integers).

### 7.2.4 What You Should Have Learned

- `//` is integer division, `%` is remainder, `**` is exponentiation
- `==` checks value, `is` checks identity (use `is` for None)
- `and` and `or` short-circuit. They do not always evaluate both sides
- `in` checks membership in strings, lists, and dictionaries
- Empty strings, zero, None, and empty collections are all falsy

## 7.3 The Gap

Operators are the glue in AI-generated code. Every condition, every calculation, every loop test uses them. Now when you see `if count % 5 == 0:` you know it checks whether count is a multiple of 5. When you see `if name and len(name) > 0:` you know the `and` short-circuits, if name is empty, it never checks the length.

In the Building Session, you will add smarter response logic to your chatbot based on message properties.

## 7.4 Building Session

### 7.4.1 The Spec

Make your chatbot respond differently based on message characteristics:

- Respond to questions (input contains "?")
- Respond differently to long messages (more than 50 characters) vs short ones
- Track whether the user has asked any questions using a boolean
- Use `in` to check for keywords

### 7.4.2 Prompt It

**i** Building Session Prompt

Update my chatbot to v0.7. Add operator-based response logic:

- If the input contains "?", treat it as a question and respond with "Good question! I'm still learning…"
- If the message is longer than 50 characters, respond with "That's a lot to take in!"
- If the message contains "python" or "coding" (use `in`), respond enthu-

siastically
- Track a boolean `has_asked_question` that becomes True when the user asks their first question
- Use the modulo operator: every 5th message, add "(milestone!)" to the response

Keep the existing features from v0.6.

### 7.4.3 Read the Code

Your AI will produce something like this:

```
"""PyBot v0.7: Operator-based responses."""
BOT_NAME = "PyBot"
VERSION = "0.7"

user_name = input(f"{BOT_NAME}: What's your name?
").strip()
while not user_name:
    user_name = input("Please enter your name:
    ").strip()

print(f"Hello, {user_name}! Type 'quit' to exit,
'help' for commands.")

message_count = 0
has_asked_question = False

while True:
    user_input = input(f"{user_name}: ").strip()

    if not user_input:
        print(f"{BOT_NAME}: Type something! Try
        'help'.")
        continue

    if user_input.lower() == "quit":
        print(f"{BOT_NAME}: Goodbye,
        {user_name}!")
        break

    message_count += 1
    milestone = " (milestone!)" if message_count %
```

```
    5 == 0 else ""

    if "?" in user_input:
        has_asked_question = True
        response = "Good question! I'm still
        learning..."
    elif "python" in user_input.lower() or
    "coding" in user_input.lower():
        response = "I love that topic!"
    elif len(user_input) > 50:
        response = "That's a lot to take in!"
    else:
        response = f"You said '{user_input}'"

    print(f"{BOT_NAME}: {response}{milestone}")
```

**What to Notice**

`"?" in user_input` uses the `in` operator for substring check. `message_count % 5 == 0` uses modulo to detect every 5th message. `has_asked_question = True` is a boolean flag. It starts `False` and flips once. The `or` in the keyword check means either match triggers the response.

### 7.4.4 Stretch It

**Building Session Prompt**

Add a "stats" command that shows: total messages, whether any questions have been asked (True/False), and the average message length (total characters // message count).

## 7.5 Your Chatbot So Far

- Ch 1-2: Basic loop, structure, comments
- Ch 3-4: Type recognition, name memory, message count
- Ch 5-6: Formatted output, input validation, help
- **Ch 7: Question detection, keyword matching, milestones**

## 7.6 Quick Reference

```
# Arithmetic
7 / 2       # 3.5 (float division)
7 // 2      # 3   (integer division)
7 % 2       # 1   (remainder)
2 ** 10     # 1024 (exponentiation)

# Comparison
x == y      # equal
x != y      # not equal
x is None   # identity (use for None)

# Logical
x and y     # both true (short-circuits)
x or y      # either true (short-circuits)
not x       # negation

# Membership
"py" in "python"        # True
5 in [1, 2, 3, 4, 5]   # True
"key" in my_dict        # True (checks keys)

# Falsy values
# False, 0, 0.0, "", [], {}, None
```

# Chapter 8

# Using Functions

## 8.1 The Wall

AI called `len()`, `range()`, `sorted()`, `enumerate()`, and `zip()` in a single block of code. You knew they were functions because of the parentheses, but you had no idea what they returned. You tried changing the arguments and got unexpected results. You looked up one function, understood it, then hit another you did not recognise.

AI uses Python's built-in functions constantly. If you do not know what is available and what each function returns, you are reading AI-generated code through fog.

This chapter fixes that.

## 8.2 Thinking Session

### 8.2.1 Getting Oriented

> **i** Thinking Session Prompt
>
> What are the most commonly used built-in functions in Python? Not all of them. Just the 10-15 that appear most often in AI-generated code. For each one, tell me what it does, what it returns, and give me a one-line example.

Your AI should cover: `print()`, `input()`, `len()`, `type()`, `int()`, `float()`, `str()`, `range()`, `list()`, `sorted()`, `enumerate()`, `zip()`, `max()`, `min()`, `sum()`. You already know some of these from earlier chapters. The new ones matter because AI uses them as building blocks.

### 8.2.2 Go Deeper

**Thinking Session Prompt**

Explain range(), enumerate(), and zip() in Python. These appear in AI-generated loops all the time and I do not fully understand them. Show me what each one produces and when I would choose one over another.

**What to Look For**

`range(5)` produces `0, 1, 2, 3, 4`. `enumerate(items)` produces `(0, "first"), (1, "second"), ...`, index and value together. `zip(list1, list2)` pairs elements from two lists. These are the three loop helpers AI reaches for constantly.

**Thinking Session Prompt**

I see AI using `.lower()`, `.strip()`, `.split()`, `.join()`, `.replace()` on strings. Are these functions or something else? What is the difference between `len(text)` and `text.lower()`?

### 8.2.3 Challenge It

**Thinking Session Prompt**

What does each line produce?

```
len("hello")
len([1, 2, 3])
list(range(3))
sorted([3, 1, 2])
sorted("python")
list(enumerate(["a", "b", "c"]))
list(zip([1, 2], ["x", "y"]))
```

**What to Look For**

`len("hello")` is 5. `len([1, 2, 3])` is 3. `list(range(3))` is `[0, 1, 2]`. `sorted([3, 1, 2])` is `[1, 2, 3]`. `sorted("python")` is `['h', 'n', 'o', 'p', 't', 'y']`. It sorts characters. `enumerate` gives `[(0, 'a'), (1, 'b'), (2, 'c')]`. `zip` gives `[(1, 'x'), (2, 'y')]`.

### 8.2.4 What You Should Have Learned

- Built-in functions are called with `function(arguments)`
- Methods are called on objects with `object.method(arguments)`
- `len()`, `type()`, `range()`, `sorted()` work on many types
- `enumerate()` and `zip()` are loop helpers you will see everywhere
- Functions return values. You can store the result or use it inline

## 8.3 The Gap

You now have a vocabulary of the built-in functions AI reaches for most often. When you see `sorted(data, key=lambda x: x["age"])` in AI-generated code, you know `sorted()` returns a new sorted list, `key` tells it what to sort by, and `lambda` creates a small anonymous function. You may not understand `lambda` fully yet, but you know what the overall line does.

In the Building Session, you will use built-in functions to add text processing to your chatbot.

## 8.4 Building Session

### 8.4.1 The Spec

Add built-in function usage to your chatbot:

- Use `len()` to measure message length
- Use `.split()` to count words
- Use `.lower()` and `.strip()` for input cleaning (already started in Ch 6)
- Track the words the user has used, in the order they first appeared
- Use a "words" command to show the 5 most recent unique words

### 8.4.2 Prompt It

**Building Session Prompt**

Update my chatbot to v0.8. Add text processing using built-in functions:

- Count the number of words in each message using split()
- For the "stats" command, show: total messages, total words (sum across all messages), average words per message
- Track every unique word the user has used in a list, keeping the words in the order they first appeared (skip a word if it is already in the list)
- For a "words" command, show the 5 most recently added unique words using list slicing

> Keep the existing features from v0.7.

### 8.4.3 Read the Code

Your AI will produce something like this:

```
"""PyBot v0.8: Built-in function processing."""
BOT_NAME = "PyBot"
VERSION = "0.8"

user_name = input(f"{BOT_NAME}: What's your name?
").strip()
while not user_name:
    user_name = input("Please enter your name:
    ").strip()

print(f"Hello, {user_name}! Type 'help' for
commands.")

message_count = 0
total_words = 0
unique_words = []

while True:
    user_input = input(f"{user_name}: ").strip()

    if not user_input:
        continue

    command = user_input.lower()

    if command == "quit":
        print(f"{BOT_NAME}: Goodbye,
        {user_name}!")
        break
    elif command == "help":
        print(f"{BOT_NAME}: Commands: quit, help,
        stats, words")
        continue
    elif command == "stats":
        avg = total_words // max(message_count, 1)
        print(f"{BOT_NAME}: Messages:
```

```
        {message_count}, "
              f"Words: {total_words}, Avg:
              {avg}/msg")
        continue
    elif command == "words":
        recent = unique_words[-5:]
        print(f"{BOT_NAME}: Recent words: {',
        '.join(recent)}")
        continue

    message_count += 1
    words = user_input.split()
    word_count = len(words)
    total_words += word_count

    for word in words:
        if word.lower() not in unique_words:
            unique_words.append(word.lower())

    print(f"{BOT_NAME}: ({word_count} words) "
          f"You said '{user_input}'")
```

**What to Notice**

`.split()` breaks a string into a list of words. `len(words)` counts them. `unique_words` starts as an empty list. The `not in` check (Chapter 7's membership operator) appends each word only the first time it appears, so `unique_words` keeps the words in the order you first used them. That order is what makes `[-5:]` mean "the 5 most recent". Slicing a list keeps its order. `max(message_count, 1)` prevents division by zero.

### 8.4.4 Stretch It

**Building Session Prompt**

Add an "info" command that takes a word as argument (e.g., "info python"). Use len() to show the word's character count, and check if it is in the unique_words list.

## 8.5 Your Chatbot So Far

- Ch 1-4: Loop, structure, types, name memory
- Ch 5-6: Formatted output, input validation
- Ch 7: Operators, keyword matching
- **Ch 8: Word counting, unique words, stats commands**

## 8.6 Quick Reference

```
# Common built-ins
len("hello")          # 5
type(42)              # <class 'int'>
range(5)              # 0, 1, 2, 3, 4
sorted([3, 1, 2])     # [1, 2, 3]
max(3, 7, 2)          # 7
min(3, 7, 2)          # 2
sum([1, 2, 3])        # 6
abs(-5)               # 5

# Loop helpers
for i, item in enumerate(items):
    print(f"{i}: {item}")

for a, b in zip(list1, list2):
    print(f"{a} -> {b}")

# String methods
"hello world".split()       # ["hello", "world"]
" ".join(["a", "b"])        # "a b"
"Hello".lower()             # "hello"
"  hi  ".strip()            # "hi"
"hello".replace("l", "L")   # "heLLo"
```

# Chapter 9

# Creating Functions

## 9.1 The Wall

AI gave you 80 lines of code in one continuous block. No functions. When you asked it to change how responses work, it rewrote the entire thing. When you found a bug, you had to trace through all 80 lines to find it. You could not reuse any part of the code because nothing was separated.

You asked AI to "add functions" and it created functions with names like `process()` and `handle()` that took too many parameters. You did not know what a good function looked like, so you could not tell whether AI's refactoring actually improved anything.

This chapter fixes that.

> **Coming from Think Python?**
>
> If you worked through *Think Python, Direct AI*, you already created functions to organise your project code. This chapter gives you the full picture, parameter types, return values, scope rules, and design principles that turn functions from something you use into something you understand.

## 9.2 Thinking Session

### 9.2.1 Getting Oriented

**Thinking Session Prompt**

Why do we use functions in Python? I understand they group code, but what is the real benefit? Explain it from the perspective of someone who mostly reads and modifies AI-generated code. When should something be a function and when is it fine as inline code?

Your AI should explain: functions make code reusable, testable, and readable. For someone reading AI code, functions create named abstractions. `get_response(message)` tells you what happens without reading the implementation. If your AI only talks about "avoiding code duplication," push for the readability and testability angles.

### 9.2.2 Go Deeper

**Thinking Session Prompt**

Explain Python function syntax step by step: def, parameters, default values, return values, docstrings. What is the difference between a parameter and an argument? What happens if a function does not have a return statement?

**What to Look For**

Key points: `def` defines a function, parameters are the names in the definition, arguments are the values passed when calling it. Default parameters use `=` in the definition. A function without `return` returns `None` implicitly. Docstrings go right after `def` as a triple-quoted string.

**Thinking Session Prompt**

What is variable scope in Python? If I define a variable inside a function, can I use it outside? What about the reverse, can a function see variables defined outside it? This confuses me in AI-generated code because sometimes functions use variables I cannot find defined inside them.

### 9.2.3 Challenge It

**Thinking Session Prompt**

What is wrong with this function, and how would you fix it?

```
def calculate_average(numbers):
    total = 0
    for n in numbers:
        total += n
    average = total / len(numbers)

result = calculate_average([10, 20, 30])
print(f"Average: {result}")
```

**What to Look For**

The function calculates the average but never returns it. `result` will be `None`. The fix is adding `return average` at the end. AI generates this bug when it forgets the return statement. The code runs without error but produces `None` instead of the actual value.

### 9.2.4 What You Should Have Learned

- `def` creates a function, `return` sends a value back
- Parameters have names; arguments are the values passed
- Default parameters make arguments optional
- Functions without `return` return `None`
- Local variables are invisible outside their function (scope)
- Good functions do one thing and have descriptive names

## 9.3 The Gap

Functions are how AI organises code, and how you should evaluate whether it organised it well. A function named `process_data()` that does five things is poorly designed. A function named `clean_user_input()` that strips whitespace and lowercases text is well designed. You can now tell the difference.

In the Building Session, you will refactor your chatbot from one long block into clean, named functions.

## 9.4 Building Session

### 9.4.1 The Spec

Refactor your chatbot into functions:

- `display_welcome()`, shows the welcome banner
- `get_user_name()`, asks for and validates the name
- `get_response(user_input, message_count)`, returns the response string
- `main()`. The main loop that ties everything together

### 9.4.2 Prompt It

> **Building Session Prompt**
>
> Refactor my chatbot to v0.9 using functions. Split the code into:
> - display_welcome(bot_name, version), prints the welcome screen
> - get_user_name(bot_name), asks for name, validates it is not empty, returns the name
> - get_response(user_input, message_count), takes the input and count, returns a response string (handles questions, keywords, numbers, default)
> - main(). The main loop that calls the other functions
>
> Each function should have a one-line docstring. Call main() at the bottom with the if **name** == "**main**" guard.
> Keep all existing features from v0.8.

### 9.4.3 Read the Code

Your AI will produce something like this:

```
"""PyBot v0.9: Refactored with functions."""
BOT_NAME = "PyBot"
VERSION = "0.9"


def display_welcome(bot_name, version):
    """Print the welcome banner."""
    print(f"Hello! I'm {bot_name} v{version}.")
    print("Type 'help' for commands.")


def get_user_name(bot_name):
```

```
    """Ask for and validate the user's name."""
    name = input(f"{bot_name}: What's your name?
    ").strip()
    while not name:
        name = input("Please enter your name:
        ").strip()
    return name


def get_response(user_input, message_count):
    """Return a response based on the input."""
    if "?" in user_input:
        return "Good question! I'm still
        learning..."
    elif "python" in user_input.lower():
        return "I love that topic!"
    try:
        number = int(user_input)
        return f"{number} is {'even' if number % 2
        == 0 else 'odd'}"
    except ValueError:
        pass
    return f"You said '{user_input}'"


def main():
    """Run the chatbot."""
    display_welcome(BOT_NAME, VERSION)
    user_name = get_user_name(BOT_NAME)
    print(f"Nice to meet you, {user_name}!")

    message_count = 0

    while True:
        user_input = input(f"{user_name}:
        ").strip()

        if not user_input:
            continue
        if user_input.lower() == "quit":
            print(f"{BOT_NAME}: Goodbye,
            {user_name}!")
```

```
            break
        if user_input.lower() == "help":
            print(f"{BOT_NAME}: Commands: quit,
            help")
            continue

        message_count += 1
        response = get_response(user_input,
        message_count)
        print(f"{BOT_NAME}: {response}")


if __name__ == "__main__":
    main()
```

**What to Notice**

Each function does one thing. `get_user_name()` returns a string. It handles its own validation loop. `get_response()` takes input and returns a response. It does not print anything. `main()` orchestrates the flow. The `if __name__ == "__main__"` guard means this file can be imported without running the chatbot, useful for testing.

### 9.4.4 Stretch It

**Building Session Prompt**

Add a `display_stats(message_count, user_name)` function that is called when the user types "stats". Also add a `display_help()` function that lists all available commands. Both should return None. They print directly.

## 9.5 Your Chatbot So Far

- Ch 1-6: Loop, structure, types, memory, formatting, validation
- Ch 7-8: Operators, built-in functions, word counting
- **Ch 9: Refactored into functions: display_welcome, get_user_name, get_response, main**

## 9.6 Quick Reference

```
# Define a function
def greet(name):
    """Greet the user by name."""
    return f"Hello, {name}!"

# Default parameters
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

# Multiple return values
def min_max(numbers):
    return min(numbers), max(numbers)
lo, hi = min_max([3, 1, 7])

# No return = returns None
def log(message):
    print(f"[LOG] {message}")

# Guard for importable scripts
if __name__ == "__main__":
    main()
```

# Chapter 10

# Making Decisions

## 10.1 The Wall

AI generated a chain of `if`, `elif`, and `else` statements. The logic was wrong. It matched the first condition when it should have matched the third. You could not trace through the conditions to find the bug because you did not understand how Python evaluates them. You rearranged the order and broke something else.

Conditional logic is where AI makes its most subtle mistakes. The code runs, the syntax is valid, but the logic is backwards. If you cannot trace through conditions yourself, you will never catch these bugs.

This chapter fixes that.

## 10.2 Thinking Session

### 10.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> Explain Python's if, elif, and else statements. I want to understand: how does Python decide which branch to take? What happens if multiple conditions are true? And what is the difference between separate if statements and an if/elif chain?

Your AI should explain that `elif` chains are mutually exclusive, only the first true condition executes. Separate `if` statements each evaluate independently. This distinction is critical: AI sometimes uses separate `if` statements when it should use `elif`, causing multiple branches to fire.

### 10.2.2 Go Deeper

**Thinking Session Prompt**

What are common patterns for conditional logic in Python? I see AI using nested if statements, ternary expressions (x if condition else y), and guard clauses. When is each pattern appropriate?

**What to Look For**

Guard clauses (early returns for special cases) flatten deeply nested code. Ternary expressions are good for simple either/or assignments but become unreadable when nested. If your AI shows deeply nested if/else, ask it to refactor with guard clauses.

**Thinking Session Prompt**

What values are "truthy" and "falsy" in Python? AI writes things like `if items:` instead of `if len(items) > 0:`. Are these the same? When does this shorthand cause problems?

### 10.2.3 Challenge It

**Thinking Session Prompt**

What does this code print, and why is it probably a bug?

```
temperature = 35
if temperature > 30:
    print("Hot")
if temperature > 20:
    print("Warm")
if temperature > 10:
    print("Mild")
```

**What to Look For**

It prints "Hot", "Warm", and "Mild", all three. The programmer probably wanted only "Hot". The fix is `elif` instead of separate `if` statements. AI generates this bug when it builds conditions incrementally without considering mutual exclusivity.

### 10.2.4 What You Should Have Learned

- `if/elif/else` chains are mutually exclusive, only one branch runs
- Separate `if` statements are independent, multiple can run
- Guard clauses simplify deeply nested logic
- Truthy/falsy: empty collections, 0, None, "" are all falsy
- Ternary: `x = "yes" if condition else "no"`

## 10.3 The Gap

When you read AI-generated conditional logic, you can now trace the flow. You know whether conditions are mutually exclusive or independent. You can spot when AI should have used `elif` instead of `if`, or when a deeply nested block could be flattened with guard clauses.

In the Building Session, you will add categorised responses to your chatbot, different behaviour for greetings, questions, farewells, and default cases.

## 10.4 Building Session

### 10.4.1 The Spec

Add response categories to your chatbot:

- Greetings: respond when input starts with "hello", "hi", "hey"
- Questions: respond when input contains "?"
- Farewells: respond when input is "bye", "goodbye", "quit"
- Default: respond to everything else
- Use an if/elif/else chain, not separate if statements

### 10.4.2 Prompt It

**Building Session Prompt**

Update the get_response() function in my chatbot to v1.0. Add categorised responses using an if/elif/else chain:

- Greetings (input starts with "hello", "hi", or "hey"): respond warmly
- Questions (input contains "?"): respond helpfully
- Farewells ("bye" or "goodbye"): respond and signal to exit
- Numbers: the existing even/odd check
- Default: a catch-all response

The function should return a tuple: (response_text, should_exit). The main loop checks should_exit to know when to break.

Update the main() function to handle the tuple return.

### 10.4.3 Read the Code

Your AI will produce something like this:

```
def get_response(user_input):
    """Categorise input and return (response,
    should_exit)."""
    text = user_input.lower().strip()

    # Check categories in priority order
    if text in ("bye", "goodbye", "quit"):
        return "Goodbye! It was nice chatting.",
        True
    elif text.startswith(("hello", "hi", "hey")):
        return "Hello! Great to hear from you.",
        False
    elif "?" in text:
        return "Good question! Let me think...",
        False
    else:
        try:
            number = int(user_input)
            parity = "even" if number % 2 == 0
            else "odd"
            return f"{number} is {parity}.", False
        except ValueError:
            return f"Interesting.
            '{user_input}'.", False
```

**What to Notice**

The `elif` chain ensures only one category matches. Farewells are checked first, if someone says "bye", that takes priority over everything else. `.startswith()` can take a tuple of options. The function returns a tuple `(response, should_exit)`. The boolean flag lets the caller decide what to do.

### 10.4.4 Stretch It

> **Building Session Prompt**
>
> Add a mood system. Track a `mood` variable that starts at "neutral". If the user sends 3 greetings in a row, mood becomes "happy". If the user sends 3 questions in a row, mood becomes "curious". Include the mood in the response.

## 10.5 Your Chatbot So Far

- Ch 1-8: Loop, types, memory, formatting, validation, operators, functions
- Ch 9: Refactored into functions
- **Ch 10: Categorised responses with if/elif/else, farewell detection**

## 10.6 Quick Reference

```
# if/elif/else (mutually exclusive)
if x > 90:
    grade = "A"
elif x > 80:
    grade = "B"
else:
    grade = "C"

# Ternary expression
status = "even" if x % 2 == 0 else "odd"

# Guard clause
def process(data):
    if not data:
        return None
    # main logic here...

# Truthy/falsy checks
if items:        # True if list is not empty
if not name:     # True if string is empty
if count:        # True if not zero
```

# Chapter 11

# Lists

## 11.1 The Wall

AI used `append()` in one place and `extend()` in another. The output had lists nested inside lists when it should have been flat. You asked AI to fix it and it replaced everything with list comprehensions you could not read. You did not know enough about lists to tell AI which operation was wrong.

Lists are Python's workhorse data structure. AI uses them in nearly every non-trivial program. If you cannot tell `append` from `extend`, or read a list comprehension, you are stuck accepting whatever AI produces.

This chapter fixes that.

> **i** Coming from Think Python?
>
> You may recognise lists from projects like the Quiz Game or Contact Book in *Think Python, Direct AI*. There you used lists to store questions or entries. This chapter explores lists more deeply, indexing, slicing, methods, and the patterns that make lists essential.

## 11.2 Thinking Session

### 11.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> Explain Python lists to me. Not just "an ordered collection". I want to understand: how do I access individual items, what happens when I try to access an item that does not exist, and what is the difference between a list and a string? They seem similar.

Your AI should cover indexing (starts at 0), negative indexing (`-1` is last item), `IndexError` for out-of-range access, and the difference: lists are mutable (you can change items), strings are not.

### 11.2.2 Go Deeper

> **Thinking Session Prompt**
>
> What are the most important list methods? I keep seeing append(), extend(), insert(), remove(), pop(), sort(), and reverse() in AI-generated code. Explain each one and, critically, tell me which ones modify the list in place versus returning a new list.

> **What to Look For**
>
> The key distinction: `append()`, `extend()`, `insert()`, `remove()`, `pop()`, `sort()`, `reverse()` all modify the list in place and return `None`. `sorted()` returns a new list. AI sometimes writes `my_list = my_list.sort()`. This sets `my_list` to `None`. If your AI does not mention this trap, ask about it.

> **Thinking Session Prompt**
>
> What is list slicing in Python? I see notation like `items[1:3]`, `items[:5]`, `items[::2]` in AI-generated code. Explain the slice notation and give me practical examples.

### 11.2.3 Challenge It

**Thinking Session Prompt**

What is wrong with each of these, and what should they be?

```
items = [1, 2, 3]
items = items.append(4)

data = [3, 1, 2]
data = data.sort()

combined = [1, 2] + 3
```

**What to Look For**

Line 2: `append()` returns `None`, so `items` becomes `None`. Fix: just `items.append(4)`. Line 5: same issue. `sort()` returns `None`. Fix: `data.sort()` alone, or `data = sorted(data)`. Line 7: cannot add an integer to a list. Fix: `[1, 2] + [3]` or `[1, 2].append(3)`.

### 11.2.4 What You Should Have Learned

- Lists are ordered, mutable sequences: `[1, 2, 3]`
- Indexing starts at 0, negative indexes count from the end
- `append()` adds one item, `extend()` adds multiple (both in place)
- `sort()` modifies in place and returns None; `sorted()` returns a new list
- Slicing: `items[start:stop:step]`
- List comprehensions: `[x * 2 for x in items]`

## 11.3 The Gap

You can now read AI-generated list operations and tell whether they are correct. When AI writes `result = items.sort()`, you know the bug. When it uses `append` where `extend` belongs, you can fix it. When it writes a list comprehension, you can parse what it does.

In the Building Session, you will add conversation history to your chatbot using lists.

## 11.4 Building Session

### 11.4.1 The Spec

Add list-based features to your chatbot:

- Store conversation history as a list of messages
- Pick random responses from a list of options
- Add a "history" command that shows recent messages
- Use list slicing to show only the last 5 messages

### 11.4.2 Prompt It

> **Building Session Prompt**
>
> Update my chatbot to v1.1. Add conversation history using lists:
> - Create a conversation_history list that stores each exchange as a string: "User: message" and "Bot: response"
> - Add a "history" command that shows the last 5 entries using list slicing
> - For the default response, use random.choice() to pick from a list of 4-5 varied responses instead of always saying the same thing
> - Add import random at the top
>
> Keep the categorised response system from v1.0.

### 11.4.3 Read the Code

Your AI will produce something like this:

```
import random

def get_response(user_input):
    """Categorise input and return response."""
    text = user_input.lower().strip()

    if text in ("bye", "goodbye", "quit"):
        return "Goodbye!", True
    elif text.startswith(("hello", "hi", "hey")):
        return random.choice([
            "Hello! Great to hear from you.",
            "Hi there! What's on your mind?",
            "Hey! How can I help?",
        ]), False
    elif "?" in text:
        return "Good question! Let me think...",
        False
```

```
    else:
        return random.choice([
            f"Interesting. '{user_input}'.",
            "Tell me more about that.",
            "I see. Go on...",
            "That's a new one for me!",
        ]), False
```

**What to Notice**

`random.choice()` picks one item from a list. This gives varied responses. The conversation history uses `append()` to add entries (in the main loop, not shown here). `history[-5:]` slices the last 5 items. Lists grow dynamically. No need to declare a size.

### 11.4.4 Stretch It

**Building Session Prompt**

Add a "search" command that takes a word and searches the conversation history for any messages containing that word. Use a list comprehension to filter matches.

## 11.5 Your Chatbot So Far

- Ch 1-9: Loop, types, memory, formatting, validation, operators, functions
- Ch 10: Categorised responses with if/elif/else
- **Ch 11: Conversation history list, random responses, history command**

## 11.6 Quick Reference

```
# Create
items = [1, 2, 3]
items = list(range(5))        # [0, 1, 2, 3, 4]

# Access
items[0]                      # first item
items[-1]                     # last item
items[1:3]                    # slice: [2, 3]
items[-3:]                    # last 3 items
```

```
# Modify (in place, return None)
items.append(4)              # add one item
items.extend([5, 6])         # add multiple
items.insert(0, "first")     # insert at index
items.remove(3)              # remove by value
items.pop()                  # remove and return
last
items.sort()                 # sort in place

# New list
sorted(items)                # returns sorted copy
[x * 2 for x in items]      # list comprehension

# Check
3 in items                   # True/False
len(items)                   # count
```

# Chapter 12

# Loops

## 12.1 The Wall

AI's loop ran forever. Or it skipped items. Or it processed the last item twice. You stared at the code, `for i in range(len(data)):`, and could not tell whether the range was right. You asked AI to fix the infinite loop and it restructured the entire program instead of fixing the one condition.

Loops are where AI makes off-by-one errors, forgets break conditions, and creates unnecessarily complex iterations. If you cannot trace through a loop manually, you will never spot these bugs.

This chapter fixes that.

## 12.2 Thinking Session

### 12.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> What are the two types of loops in Python, for and while? When should I use each one? And why does AI-generated code sometimes use `for i in range(len(items)):` instead of just `for item in items:`? Is the longer version ever better?

Your AI should explain: `for` iterates over a sequence (use when you know what you are iterating over), `while` repeats until a condition is false (use when you do not know how many times). The `range(len())` pattern gives you the index, sometimes needed, but `enumerate()` is better for that.

### 12.2.2 Go Deeper

**Thinking Session Prompt**

Explain break, continue, and else on loops in Python. I see AI using all three and I do not understand the else on a for loop. That seems strange. When would you actually use it?

**What to Look For**

`break` exits the loop immediately. `continue` skips to the next iteration. The `else` on a loop runs only if the loop completed without `break`. It is useful for "search" patterns where you want to know if you found something. If your AI cannot explain a practical use case for loop `else`, the concept is probably not important enough to use.

**Thinking Session Prompt**

What are common loop patterns in Python? I see AI using counting loops, accumulator loops, search loops, and filter loops. Show me one example of each and tell me when to recognise each pattern.

### 12.2.3 Challenge It

**Thinking Session Prompt**

What is wrong with each of these loops?

```
# Loop 1
while True:
    data = input("Enter data: ")
    process(data)

# Loop 2
items = [1, 2, 3, 4, 5]
for i in range(len(items)):
    if items[i] % 2 == 0:
        items.remove(items[i])

# Loop 3
total = 0
for i in range(10):
    total = i
```

What to Look For

Loop 1: no `break`. It runs forever. Loop 2: modifying a list while iterating over it skips items (removing index 1 shifts everything, so index 2 becomes the old index 3). Loop 3: `total = i` overwrites instead of accumulating, should be `total += i`. These are all bugs AI generates regularly.

### 12.2.4 What You Should Have Learned

- `for` iterates over sequences, `while` repeats on a condition
- `break` exits, `continue` skips, `else` runs if no break
- Do not modify a list while iterating over it
- Use `enumerate()` instead of `range(len())`
- Common patterns: counting, accumulating, searching, filtering
- `while True` needs a `break` condition

## 12.3 The Gap

You can now trace through any loop AI generates and predict its behaviour. You know the difference between accumulating (+=) and overwriting (=). You know why modifying a list during iteration is dangerous. When AI gives you a loop, you can verify it will terminate and process every item exactly once.

In the Building Session, you will add word processing to your chatbot, iterating through user input to extract meaning.

## 12.4 Building Session

### 12.4.1 The Spec

Add loop-based text processing to your chatbot:

- Count specific keywords using a for loop
- Process each word in the user's message
- Use a while loop for multi-step interactions (e.g., a quiz)
- Add a "quiz" command that asks 3 Python questions in sequence

### 12.4.2 Prompt It

> **Building Session Prompt**
>
> Update my chatbot to v1.2. Add loop-based features:
> - In get_response(), use a for loop to count how many "positive" words (good, great, nice, thanks, awesome) appear in the input
> - If 2+ positive words found, respond with extra enthusiasm
> - Add a "quiz" command that uses a while loop to ask 3 Python trivia questions, tracks the score, and shows results at the end
> - The quiz should use a list of (question, answer) tuples
>
> Keep existing features.

### 12.4.3 Read the Code

Your AI will produce something like this:

```
POSITIVE_WORDS = ["good", "great", "nice",
"thanks", "awesome"]

QUIZ_QUESTIONS = [
    ("What function prints output?", "print"),
    ("What type is 3.14?", "float"),
    ("What keyword exits a loop?", "break"),
]


def count_positive(text):
    """Count positive words in text."""
    words = text.lower().split()
    count = 0
    for word in words:
```

```
        if word in POSITIVE_WORDS:
            count += 1
    return count


def run_quiz(bot_name):
    """Run a Python trivia quiz."""
    score = 0
    question_num = 0

    while question_num < len(QUIZ_QUESTIONS):
        question, answer =
        QUIZ_QUESTIONS[question_num]
        user_answer = input(f"Q{question_num + 1}:
        {question} ").strip().lower()

        if user_answer == answer:
            print(f"{bot_name}: Correct!")
            score += 1
        else:
            print(f"{bot_name}: The answer was
            '{answer}'.")

        question_num += 1

    print(f"{bot_name}: Quiz done! Score:
    {score}/{len(QUIZ_QUESTIONS)}")
```

**What to Notice**

`count_positive()` uses a for loop with an accumulator pattern (`count += 1`). `run_quiz()` uses a while loop because the number of iterations depends on the question list length. The tuple unpacking `question, answer = QUIZ_QUESTIONS[question_num]` is a pattern AI uses frequently.

### 12.4.4 Stretch It

> **Building Session Prompt**
>
> Rewrite count_positive() as a list comprehension that filters positive words, then use len() on the result. Compare both versions, which is more readable?

## 12.5 Your Chatbot So Far

- Ch 1-10: Full feature set with functions and categorised responses
- Ch 11: Conversation history, random responses
- **Ch 12: Word counting loop, quiz with while loop**

## 12.6 Quick Reference

```
# for loop
for item in items:
    print(item)

for i, item in enumerate(items):
    print(f"{i}: {item}")

# while loop
while condition:
    # do something
    if done:
        break

# Loop control
break          # exit loop
continue       # skip to next iteration

# Patterns
# Accumulate
total = 0
for x in numbers:
    total += x

# Filter
evens = [x for x in numbers if x % 2 == 0]
```

```
# Search
for item in items:
    if item == target:
        print("Found!")
        break
```

# Chapter 13

# Strings

## 13.1 The Wall

AI's string slicing gave you the wrong substring. It used `text[3:7]` and you expected 4 characters but got something different. In another case, AI used `.split()`, `.join()`, `.replace()`, and `.strip()` in a chain so long you could not tell what the final result would be without running it.

Strings are the most common data type in AI-generated code, every user interaction, every API response, every file path is a string. If you cannot predict what string operations do, you cannot debug half the code AI produces.

This chapter fixes that.

> **i** Coming from Think Python?
>
> If you worked through *Think Python, Direct AI*, you already used string methods like `.lower()` and `.split()` to process user input in your projects. Here we explore strings more thoroughly, how they work internally, the full range of methods, and when to reach for each one.

## 13.2 Thinking Session

### 13.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> Explain Python strings in depth. I know they are text in quotes, but I want to understand: are they like lists? Can I change individual characters? How does indexing and slicing work on strings compared to lists?

Your AI should explain that strings are immutable sequences. You can index and slice them like lists, but you cannot change individual characters. `text[0] = "X"` fails. To modify a string, you create a new one. This matters because AI sometimes writes code that tries to modify strings in place.

### 13.2.2 Go Deeper

> **Thinking Session Prompt**
>
> What are the most useful string methods in Python? Focus on the ones that AI uses most: split(), join(), strip(), replace(), startswith(), endswith(), find(), upper(), lower(), count(). For each one, show me what it does and what it returns.

> **What to Look For**
>
> Key pattern: all string methods return new strings (strings are immutable). `text.split()` returns a list. `" ".join(words)` returns a string. `text.replace("old", "new")` returns a new string. The original is unchanged. If your AI implies these modify the original string, that is wrong.

> **Thinking Session Prompt**
>
> Explain string slicing in Python. What does `text[2:5]` give me? What about `text[::-1]`? I want to be able to look at any slice notation in AI-generated code and predict the result.

### 13.2.3 Challenge It

**Thinking Session Prompt**

What does each line produce?

```
"hello world".split()
" ".join(["hello", "world"])
"  hello  ".strip()
"hello world".replace("world", "Python")
"hello"[1:4]
"hello"[::-1]
"hello world".title()
"python".count("o")
```

**What to Look For**

`split()` gives `["hello", "world"]`. `join()` gives `"hello world"`. `strip()` gives `"hello"`. `replace()` gives `"hello Python"`. `[1:4]` gives `"ell"` (index 1, 2, 3, stop is exclusive). `[::-1]` gives `"olleh"` (reversed). `title()` gives `"Hello World"`. `count("o")` gives `1`.

### 13.2.4 What You Should Have Learned

- Strings are immutable, all methods return new strings
- Indexing and slicing work like lists: `text[start:stop:step]`
- `split()` makes a list, `join()` makes a string
- `strip()` removes whitespace, `replace()` substitutes text
- `startswith()` and `endswith()` are cleaner than slicing for checks
- f-strings embed expressions: `f"Hello {name}"`

## 13.3 The Gap

Strings are how your chatbot processes everything the user types. Every message is a string. Every response is built from strings. Now you can read AI-generated string operations and predict the output. When AI chains five string methods together, you can trace through each step.

In the Building Session, you will add pattern matching to your chatbot using string methods.

## 13.4 Building Session

### 13.4.1 The Spec

Add string-based pattern matching to your chatbot:

- Detect greetings using `startswith()`
- Extract topics from messages using `split()` and keyword detection
- Format responses with proper capitalisation using `title()` and `capitalize()`
- Add a "repeat" command that echoes the last message in different formats (upper, lower, title, reversed)

### 13.4.2 Prompt It

**Building Session Prompt**

Update my chatbot to v1.3. Enhance the text processing:

- Use startswith() for greeting detection instead of checking exact matches
- Add a process_message() function that cleans input (strip, normalise whitespace) and extracts keywords using split()
- Add a "repeat" command that shows the last user message in: UPPER, lower, Title Case, and reversed
- Format bot responses so the first letter is always capitalised

Keep existing features.

### 13.4.3 Read the Code

Your AI will produce something like this:

```
def process_message(text):
    """Clean and analyse a message."""
    cleaned = " ".join(text.split())  # normalise
    whitespace
    words = cleaned.lower().split()
    return cleaned, words


def format_repeat(text):
    """Show text in multiple formats."""
    lines = [
        f"  UPPER:    {text.upper()}",
```

```
        f"  lower:    {text.lower()}",
        f"  Title:    {text.title()}",
        f"  Reversed: {text[::-1]}",
    ]
    return "\n".join(lines)
```

> **What to Notice**
>
> `" ".join(text.split())` is a common idiom. It splits on any whitespace and rejoins with single spaces, normalising " hello world " to "hello world". `text[::-1]` reverses the string. `"\n".join(lines)` joins a list of strings with newlines between them. All these methods return new strings. The original is never modified.

### 13.4.4 Stretch It

> **Building Session Prompt**
>
> Add a "find" command that takes a word and searches the conversation history for messages containing that word. Show the matching messages with the search word highlighted by wrapping it in asterisks using replace().

## 13.5 Your Chatbot So Far

- Ch 1-11: Full features, functions, history, random responses
- Ch 12: Loop-based processing, quiz
- **Ch 13: String pattern matching, text normalisation, repeat command**

## 13.6 Quick Reference

```
# String methods (return new strings)
"hello world".split()           # ["hello",
"world"]
" ".join(["a", "b"])            # "a b"
"  hi  ".strip()                # "hi"
"hello".replace("l", "L")      # "heLLo"
"hello".upper()                 # "HELLO"
"HELLO".lower()                 # "hello"
"hello world".title()           # "Hello World"
```

```
"hello".startswith("he")        # True
"hello".endswith("lo")          # True
"hello".find("ll")              # 2 (index)
"hello".count("l")              # 2

# Slicing
text = "Python"
text[0]          # "P"
text[1:4]        # "yth"
text[::-1]       # "nohtyP"

# f-strings
f"Hello {name}"
f"{value:.2f}"
f"{'centred':^20}"
```

# Chapter 14

# Dictionaries

## 14.1 The Wall

AI used a dictionary when a list would have worked, and you did not know enough to question it. In another case, your code crashed with `KeyError: 'username'` because the dictionary did not have the key AI assumed existed. You had no idea how to check whether a key exists before accessing it.

Dictionaries are how AI organises structured data, user profiles, configuration, API responses, keyword mappings. If you cannot read dictionary operations, you cannot work with most real-world AI-generated code.

This chapter fixes that.

> **Coming from Think Python?**
>
> You may recognise dictionaries from the Contact Book project in *Think Python, Direct AI*, where you used them to store names and phone numbers. Here we explore dictionaries more thoroughly, how they work, the full range of methods, and the patterns that make them essential.

## 14.2 Thinking Session

### 14.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> Explain Python dictionaries. How are they different from lists? When should I use a dictionary instead of a list? And what actually happens when I write `user = {"name": "Alice", "age": 30}`, how does Python find values by key?

Your AI should explain: dictionaries map keys to values, lists map indexes to values. Use a dictionary when you need to look things up by name, not by position. Keys must be immutable (strings, numbers, tuples) and unique. If your AI does not mention that dictionaries are unordered in older Python but ordered by insertion in Python 3.7+, that is a useful detail.

### 14.2.2 Go Deeper

> **Thinking Session Prompt**
>
> What are the most important dictionary methods? I see AI using get(), keys(), values(), items(), update(), and pop(). Explain each one. Specifically, what is the difference between `user["name"]` and `user.get("name")`, and why does AI sometimes use one versus the other?

> **What to Look For**
>
> The critical difference: `user["name"]` raises `KeyError` if the key does not exist. `user.get("name")` returns `None` (or a default you specify). AI should use `.get()` for keys that might not exist. If your AI does not emphasise this, it is the single most important dictionary concept.

> **Thinking Session Prompt**
>
> How do I loop through a dictionary in Python? What do I get when I use a for loop directly on a dictionary? And what is the items() method for?

### 14.2.3 Challenge It

**Thinking Session Prompt**

What happens with each of these?

```
data = {"a": 1, "b": 2}
data["c"]
data.get("c")
data.get("c", 0)
"a" in data
1 in data
data["a"] = 10
data.update({"b": 20, "d": 4})
```

**What to Look For**

`data["c"]` raises `KeyError`. `data.get("c")` returns `None`. `data.get("c", 0)` returns `0`. `"a" in data` is `True` (checks keys). `1 in data` is `False` (1 is a value, not a key). `data["a"] = 10` updates the value. `data.update(...)` merges two dictionaries.

### 14.2.4 What You Should Have Learned

- Dictionaries map keys to values: `{"key": value}`
- Use `dict[key]` when you know the key exists, `.get(key)` when you do not
- `in` checks keys, not values
- `items()` gives key-value pairs for looping
- Keys must be immutable; values can be anything
- Dictionaries are mutable. You can add, change, and remove entries

## 14.3 The Gap

Dictionaries are how your chatbot will map keywords to responses, instead of long if/elif chains, you look up the response by category. This is the pattern AI uses in most structured programs: configuration as dictionaries, response templates as dictionaries, user data as dictionaries.

In the Building Session, you will replace the if/elif response logic with a dictionary-based approach.

## 14.4 Building Session

### 14.4.1 The Spec

Restructure your chatbot's responses using dictionaries:

- Map response categories to lists of possible responses
- Map keywords to categories
- Use `.get()` with a default for unknown categories
- Add a "topics" command that shows all known categories

### 14.4.2 Prompt It

> **Building Session Prompt**
>
> Update my chatbot to v1.4. Replace the if/elif response chain with dictionaries:
> - Create a RESPONSES dict mapping categories ("greeting", "question", "farewell", "default") to lists of response strings
> - Create a KEYWORDS dict mapping trigger words to categories (e.g., "hello" □ "greeting", "bye" □ "farewell")
> - Write a classify_input(text) function that checks each word against KEYWORDS and returns the category (default to "default")
> - Update get_response() to use the dictionaries instead of if/elif
> - Add a "topics" command that shows all response categories and their keyword triggers
>
> Keep existing features.

### 14.4.3 Read the Code

Your AI will produce something like this:

```
RESPONSES = {
    "greeting": [
        "Hello! Great to hear from you.",
        "Hi there! What's on your mind?",
    ],
    "question": [
        "Good question! Let me think...",
        "That's interesting. I'm learning too.",
    ],
    "farewell": [
        "Goodbye! It was nice chatting.",
```

```
        "See you later!",
    ],
    "default": [
        "Tell me more about that.",
        "Interesting! Go on...",
    ],
}

KEYWORDS = {
    "hello": "greeting",
    "hi": "greeting",
    "hey": "greeting",
    "bye": "farewell",
    "goodbye": "farewell",
    "quit": "farewell",
}


def classify_input(text):
    """Map input to a response category."""
    words = text.lower().split()
    for word in words:
        if word in KEYWORDS:
            return KEYWORDS[word]
    if "?" in text:
        return "question"
    return "default"


def get_response(user_input):
    """Get a response using dictionary lookup."""
    category = classify_input(user_input)
    responses = RESPONSES.get(category,
    RESPONSES["default"])
    return random.choice(responses), category ==
    "farewell"
```

**What to Notice**

The if/elif chain is gone, replaced by dictionary lookups. Adding a new response category means adding entries to `RESPONSES` and `KEYWORDS`, not mod-

ifying logic. `RESPONSES.get(category, RESPONSES["default"])` uses `.get()` with a fallback. This is how AI structures most configurable systems.

### 14.4.4 Stretch It

**Building Session Prompt**

Add a "learn" command that lets the user teach the bot a new keyword-category mapping. Store it in the KEYWORDS dictionary. Test it by teaching a new keyword and then using it.

## 14.5 Your Chatbot So Far

- Ch 1-12: Full features with functions, loops, text processing
- Ch 13: String pattern matching
- **Ch 14: Dictionary-based responses, keyword classification, configurable categories**

## 14.6 Quick Reference

```
# Create
user = {"name": "Alice", "age": 30}
empty = {}

# Access
user["name"]                # "Alice" (KeyError if
missing)
user.get("name")            # "Alice" (None if
missing)
user.get("role", "guest")   # "guest" (default)

# Modify
user["email"] = "a@b.com"   # add/update
user.update({"age": 31})    # merge
del user["age"]              # delete
user.pop("age", None)        # delete (no error if
missing)

# Check
```

```
"name" in user                # True (checks keys)

# Loop
for key in user:              # keys only
for key, value in user.items():  # both
for value in user.values():      # values only

# Comprehension
squares = {x: x**2 for x in range(5)}
```

# Chapter 15

# Files

## 15.1 The Wall

AI wrote code that saved data to a file. You ran it, saw "Data saved!", and felt good. Then you restarted the program and the data was gone. It turned out AI opened the file with `"w"` (write mode) instead of `"a"` (append mode), so every run erased the previous data.

In another case, AI read a file but forgot to handle the case where the file did not exist. The program crashed with `FileNotFoundError`, on the user's machine, not yours, because the file was in your test folder.

This chapter fixes that.

## 15.2 Thinking Session

### 15.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> How does file I/O work in Python? Explain the open/read/write/close pattern. What are the different file modes (r, w, a, x) and when would I use each? Why does AI-generated code always use `with open()` instead of calling open() and close() separately?

Your AI should explain that `with open()` is a context manager that automatically closes the file even if an error occurs. The modes matter: `"r"` reads, `"w"` writes (erases existing content), `"a"` appends, `"x"` creates (fails if file exists). AI should always use `with`, if it does not, that is a code smell.

### 15.2.2 Go Deeper

**Thinking Session Prompt**

What is the difference between reading a file with read(), readline(), and readlines()? And for writing, what is the difference between write() and writelines()? When does AI use each one?

**What to Look For**

`read()` returns the entire file as one string. `readline()` returns one line. `readlines()` returns a list of lines. For writing, `write()` takes a string, `writelines()` takes a list of strings (and does not add newlines). AI usually uses `read()` for small files and iterates line by line for large ones.

**Thinking Session Prompt**

How do I work with JSON files in Python? AI-generated code often saves configuration and data as JSON. Show me how to read and write JSON files using the json module.

### 15.2.3 Challenge It

**Thinking Session Prompt**

What is wrong with this code?

```
f = open("data.txt", "w")
f.write("Hello")
# program crashes here
f.close()
```

**What to Look For**

If the program crashes between `open()` and `close()`, the file is never properly closed, potentially losing data. The fix is `with open("data.txt", "w") as f:`. The `with` block guarantees closure. Also, `"w"` mode erases existing content, if that is not intended, use `"a"`.

### 15.2.4 What You Should Have Learned

- Always use `with open()`. It handles closing automatically

- File modes: `"r"` read, `"w"` write (erases), `"a"` append
- `read()` gets all content, iterate for line-by-line processing
- JSON is the standard format for structured data: `json.load()` / `json.dump()`
- Always handle `FileNotFoundError` when reading files

## 15.3 The Gap

Files give your chatbot memory that survives between runs. Without files, everything resets when the program stops. Now you know how to read, write, and append, and how to use JSON for structured data. You also know the most common file bug AI generates: wrong file mode.

In the Building Session, you will give your chatbot persistent memory.

## 15.4 Building Session

### 15.4.1 The Spec

Add file persistence to your chatbot:

- Save conversation history to a JSON file after each session
- Load previous conversation history when the chatbot starts
- Handle the case where the history file does not exist yet
- Add a "save" command to save mid-conversation

### 15.4.2 Prompt It

**Building Session Prompt**

Update my chatbot to v1.5. Add file-based persistence:

- Save conversation history to "chat_history.json" using json.dump()
- Load previous history on startup using json.load() with FileNotFoundError handling
- The history should be a list of dicts: [{"role": "user", "text": "..."}, {"role": "bot", "text": "..."}]
- Add a "save" command that saves the current history
- Auto-save when the user quits
- Add import json at the top

Keep existing dictionary-based response system from v1.4.

### 15.4.3 Read the Code

Your AI will produce something like this:

```
import json

HISTORY_FILE = "chat_history.json"


def load_history():
    """Load conversation history from file."""
    try:
        with open(HISTORY_FILE, "r") as f:
            return json.load(f)
    except FileNotFoundError:
        return []


def save_history(history):
    """Save conversation history to file."""
    with open(HISTORY_FILE, "w") as f:
        json.dump(history, f, indent=2)
```

**What to Notice**

`json.load()` reads structured data from a file. `json.dump()` writes it. `indent=2` makes the JSON human-readable. The `try/except` `FileNotFoundError` handles the first run when no history file exists yet. `"w"` mode is correct here because we write the entire history each time (not appending).

### 15.4.4 Stretch It

**Building Session Prompt**

Add a "history" command that loads and displays the last 10 entries from the saved file, formatted with timestamps. Use the datetime module to add timestamps when saving each entry.

## 15.5 Your Chatbot So Far

- Ch 1-14: Full features with dictionaries, loops, string processing

- **Ch 15: Persistent conversation history saved to JSON file**

## 15.6 Quick Reference

```
# Read a file (pick ONE way to read it per open)
with open("file.txt", "r") as f:
    content = f.read()          # entire file as
    one string

with open("file.txt", "r") as f:
    lines = f.readlines()       # list of lines

# A handle reads once: after f.read(),
f.readlines() returns []
# Reopen the file if you need to read it a second
time

# Write a file
with open("file.txt", "w") as f:    # erases first
    f.write("hello\n")

# Append to a file
with open("file.txt", "a") as f:    # keeps
existing
    f.write("more\n")

# JSON
import json
with open("data.json", "r") as f:
    data = json.load(f)
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)

# Handle missing file
try:
    with open("file.txt") as f:
        data = f.read()
except FileNotFoundError:
    data = ""
```

# Chapter 16

# Errors and Exceptions

## 16.1 The Wall

AI wrapped everything in `try: ... except: pass` and your errors vanished. The program did not crash anymore, but it also did not work. It silently swallowed every problem. You had no idea where things went wrong because the error messages were being hidden.

Or the opposite: AI did not add error handling at all, and the program crashed on the first unexpected input with a traceback you could not read.

This chapter fixes that.

## 16.2 Thinking Session

### 16.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> What is the difference between a syntax error and an exception in Python? When I see a traceback, what information does it give me? Walk me through how to read a Python error message. I want to be able to look at AI-generated tracebacks and immediately find the problem.

Your AI should explain: syntax errors are caught before the code runs (missing colons, bad indentation), exceptions happen during execution (dividing by zero, missing keys, wrong types). The traceback reads bottom-up: the last line is the error, the lines above show the call stack. The file name and line number tell you exactly where to look.

### 16.2.2 Go Deeper

**Thinking Session Prompt**

What are the most common Python exceptions? I see TypeError, ValueError, KeyError, IndexError, FileNotFoundError, and AttributeError in AI-generated code. For each one, tell me what causes it and show me a one-line example that triggers it.

**What to Look For**

`TypeError`: wrong type for an operation (`"5" + 3`). `ValueError`: right type but wrong value (`int("hello")`). `KeyError`: missing dictionary key (`d["missing"]`). `IndexError`: list index out of range (`[1,2][5]`). `FileNotFoundError`: file does not exist. `AttributeError`: calling a method that does not exist on that type.

**Thinking Session Prompt**

Explain Python's try/except properly. What is the right way to use it? Why is `except:` or `except Exception:` without specifying the error type considered bad practice? What is the finally clause for?

### 16.2.3 Challenge It

**Thinking Session Prompt**

What is wrong with this error handling?

```
try:
    data = json.load(open("config.json"))
    result = data["setting"] / data["count"]
    save_to_database(result)
except:
    pass
```

**What to Look For**

Three problems: (1) bare `except:` catches everything including KeyboardInterrupt and SystemExit. (2) `pass` silently swallows errors. You will never know something went wrong. (3) the try block is too broad. It should be

separate try/excepts for file reading, data access, and database saving. AI generates this pattern constantly.

### 16.2.4 What You Should Have Learned

- Syntax errors vs exceptions, different causes, different fixes
- How to read a traceback: bottom-up, file + line number
- The common exceptions: TypeError, ValueError, KeyError, IndexError
- try/except should catch specific exceptions, not bare except
- Never use `except: pass`, at minimum, log the error
- `finally` runs whether or not an exception occurred

## 16.3 The Gap

You can now read Python error messages and understand what went wrong. When AI generates error handling, you can evaluate whether it is catching the right exceptions or blindly swallowing everything. This is a critical skill, bad error handling is worse than no error handling.

In the Building Session, you will add proper error handling to your chatbot.

## 16.4 Building Session

### 16.4.1 The Spec

Add robust error handling to your chatbot:

- Handle `FileNotFoundError` when loading conversation history
- Handle `json.JSONDecodeError` when the history file is corrupted
- Handle `KeyboardInterrupt` (Ctrl+C) gracefully
- Log errors instead of silently ignoring them

### 16.4.2 Prompt It

**i** Building Session Prompt

Update my chatbot to v1.6. Add proper error handling:

- Wrap the history file loading in try/except that handles FileNotFoundError and json.JSONDecodeError separately, with specific error messages for each
- Add a try/except around the main loop that catches KeyboardInterrupt and saves history before exiting

- Add an error_log list that stores error messages during the session
- Add an "errors" command that shows any errors that occurred
- Never use bare except: or except: pass

Keep existing features.

### 16.4.3 Read the Code

Your AI will produce something like this:

```
def load_history():
    """Load history with specific error
    handling."""
    try:
        with open(HISTORY_FILE, "r") as f:
            return json.load(f)
    except FileNotFoundError:
        print(f"{BOT_NAME}: No previous history
        found. "
              "Starting fresh.")
        return []
    except json.JSONDecodeError:
        print(f"{BOT_NAME}: History file
        corrupted. "
              "Starting fresh.")
        return []


def main():
    """Main loop with error handling."""
    display_welcome(BOT_NAME, VERSION)
    user_name = get_user_name(BOT_NAME)
    history = load_history()
    error_log = []

    try:
        while True:
            user_input = input(f"{user_name}:
            ").strip()
            if not user_input:
                continue
            # ... response logic ...
    except KeyboardInterrupt:
```

```
        print(f"\n{BOT_NAME}: Interrupted!
        Saving...")
    finally:
        save_history(history)
        print(f"{BOT_NAME}: History saved.
        Goodbye!")
```

**What to Notice**

Each `except` catches a specific exception with a helpful message. `KeyboardInterrupt` is caught separately. It is not a bug, it is a user action. The `finally` block saves history regardless of how the loop ends (quit command, Ctrl+C, or unexpected error). Errors are logged to a list, not silently discarded.

### 16.4.4 Stretch It

**Building Session Prompt**

Add a custom exception class `ChatbotError` that is raised when the chatbot encounters an invalid command format. Catch it in the main loop and add the error to the error_log.

## 16.5 Your Chatbot So Far

- Ch 1-14: Full features with dictionaries and string processing
- Ch 15: File persistence
- **Ch 16: Proper error handling, error logging, graceful shutdown**

## 16.6 Quick Reference

```
# Specific exception handling
try:
    result = int(user_input)
except ValueError:
    print("Not a valid number")
except TypeError:
    print("Wrong type")
```

```
# Multiple exceptions
except (ValueError, TypeError) as e:
    print(f"Error: {e}")

# finally always runs
try:
    f = open("file.txt")
except FileNotFoundError:
    print("Missing file")
finally:
    print("This always runs")

# Raise your own
raise ValueError("Invalid input")

# Custom exception
class ChatbotError(Exception):
    pass
```

# Chapter 17

# Debugging

## 17.1 The Wall

You pasted an error message into AI and it gave you a "fix" that introduced a different error. You pasted that error in and got yet another fix. Three rounds later, the code was completely different from what you started with and you had no idea what changed or why.

The problem was not AI's debugging ability. It was that you could not isolate the bug yourself. You gave AI the entire program and said "fix it" instead of identifying which specific part was failing. AI cannot debug effectively without precise context, and you could not provide that context.

This chapter fixes that.

## 17.2 Thinking Session

### 17.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> What are the main debugging strategies in Python? I know I can paste errors into AI, but I want to understand how to narrow down where a bug is before I ask for help. What is print debugging, what is the Python debugger (pdb), and when would I use each?

Your AI should explain: print debugging (adding `print()` statements to trace values) is quick and works everywhere. `pdb` (Python debugger) lets you step through code line by line. The strategy matters more than the tool: isolate the bug first, then fix it.

### 17.2.2 Go Deeper

**Thinking Session Prompt**

When I have a bug, what is a systematic process for finding it? Not just "add print statements everywhere", give me a step-by-step method. How do I narrow down which function, which line, which variable is causing the problem?

**What to Look For**

A good debugging process: (1) reproduce the bug reliably, (2) form a hypothesis about what is wrong, (3) test the hypothesis by checking values at specific points, (4) fix the smallest thing possible, (5) verify the fix did not break something else. This is scientific method applied to code.

**Thinking Session Prompt**

How should I use AI for debugging effectively? What information should I include when I ask AI to help me fix a bug? What is the difference between "fix my code" and a good debugging prompt?

### 17.2.3 Challenge It

**Thinking Session Prompt**

This code is supposed to count vowels in a string but returns the wrong number. Debug it, do not just fix it, explain how you would find the bug step by step.

```
def count_vowels(text):
    vowels = "aeiou"
    count = 0
    for char in text:
        if char in vowels:
            count += 1
    return count

result = count_vowels("Hello World")
print(f"Vowels: {result}")  # Expected: 3, Got: 3
```

What to Look For

Trick question. This code is actually correct for lowercase vowels. But it misses uppercase vowels. `"Hello World"` has 3 lowercase vowels (e, o, o) and 0 uppercase. If the expected count was 3, the code works. But if the user expected to count "E" too, the fix is `text.lower()` or `vowels = "aeiouAEIOU"`. The lesson: always clarify what "correct" means.

### 17.2.4 What You Should Have Learned

- Debugging is a process: reproduce, hypothesise, test, fix, verify
- Print debugging: add `print(f"DEBUG: {variable=}")` at key points
- `pdb`: `breakpoint()` in your code to pause and inspect
- When asking AI for help: include the error, the relevant code, what you expected, and what you tried
- Do not ask AI to "fix my code", ask it to explain why a specific line produces a specific result

## 17.3 The Gap

You now have a debugging process that works with or without AI. When you encounter a bug, you can narrow it down to a specific function, a specific line, a specific variable. When you do ask AI for help, you can provide precise context instead of dumping the entire program.

In the Building Session, you will add a debug mode to your chatbot.

## 17.4 Building Session

### 17.4.1 The Spec

Add a debug mode to your chatbot:

- A `--debug` flag or "debug" command that toggles verbose output
- In debug mode, show: the raw input, the classified category, the selected response, and the current history length
- Debug output should be visually distinct from normal output

### 17.4.2 Prompt It

> **Building Session Prompt**
>
> Update my chatbot to v1.7. Add a debug mode:
> - Add a DEBUG constant (default False)
> - Add a "debug" command that toggles debug mode on/off
> - When debug mode is on, after each message print: [DEBUG] Raw input: "…" [DEBUG] Category: "greeting" [DEBUG] History length: 15
> - Use a debug_log() function that only prints when DEBUG is True
> - Debug output should use [DEBUG] prefix to distinguish from normal output
>
> Keep existing features.

### 17.4.3 Read the Code

Your AI will produce something like this:

```
debug_mode = False


def debug_log(*args):
    """Print debug info only when debug mode is
    on."""
    if debug_mode:
        print("[DEBUG]", *args)


# Inside main(), before the loop:
    global debug_mode  # without this, the
    assignment below creates a local

    # In the main loop:
    if command == "debug":
        debug_mode = not debug_mode
        status = "ON" if debug_mode else "OFF"
        print(f"{BOT_NAME}: Debug mode {status}")
        continue

    category = classify_input(user_input)
    debug_log(f"Raw: '{user_input}'")
    debug_log(f"Category: {category}")
```

```
    debug_log(f"History: {len(history)} entries")

    response, should_exit =
    get_response(user_input)
    debug_log(f"Response: '{response}'")
```

What to Notice

`debug_log()` uses `*args` to accept any number of arguments, just like `print()`. The `[DEBUG]` prefix makes debug output instantly recognisable. The toggle uses `not debug_mode` to flip the boolean. And `global debug_mode` is not decoration: without it, the assignment inside `main()` creates a local variable, `debug_log()` keeps reading the untouched global, and the toggle silently does nothing. This is the same scope trap Chapter 9's Challenge demonstrated, and a bug AI produces regularly. A function that conditionally prints is how professional code handles debug output.

### 17.4.4 Stretch It

Building Session Prompt

Add timing to the debug output. Use `import time` and `time.time()` to measure how long each response takes to generate. Show it as "[DEBUG] Response time: 0.002s".

## 17.5 Your Chatbot So Far

- Ch 1-15: Full features with file persistence
- Ch 16: Error handling and logging
- **Ch 17: Debug mode with toggle and verbose output**

## 17.6 Quick Reference

```
# Print debugging
print(f"DEBUG: {variable=}")        # Python 3.8+
print(f"DEBUG: x={x}, y={y}")

# Python debugger
breakpoint()  # pauses execution here
```

```
# Commands: n (next), s (step in), c (continue),
#           p variable (print), q (quit)

# Conditional debug output
def debug_log(*args, enabled=False):
    if enabled:
        print("[DEBUG]", *args)

# Assert (catches impossible states)
assert len(items) > 0, "List should not be empty"
```

# Chapter 18

# Testing

## 18.1 The Wall

AI said the code worked. You ran it, it seemed fine. Then you gave it to someone else and it crashed with inputs you never tried. AI tested its own code with the happy path. The obvious, expected inputs, and missed every edge case.

You asked AI to "add tests" and it generated tests that passed trivially. One test checked that `add(2, 3)` returns `5`. It did not test what happens when you pass `None`, a string, or a negative number. The tests gave you false confidence.

This chapter fixes that.

## 18.2 Thinking Session

### 18.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> Why do we test code? I know AI can generate tests, but what makes a test actually useful? What is the difference between a test that gives you confidence and a test that just takes up space? And what is pytest, why do Python developers prefer it over the built-in unittest module?

Your AI should explain: good tests check edge cases, not just happy paths. A test is useful if its failure tells you something is broken. pytest is preferred because it uses plain `assert` statements instead of special methods (`assertEqual`, `assertTrue`), making tests more readable.

### 18.2.2 Go Deeper

**Thinking Session Prompt**

How do I write a pytest test? Walk me through the structure: where do test files go, how do I name them, what does a test function look like, and how do I run tests? Also explain what "arrange, act, assert" means.

**What to Look For**

Test files go in a `tests/` directory, named `test_*.py`. Test functions start with `test_`. Arrange-Act-Assert: set up the input (arrange), call the function (act), check the result (assert). `pytest` discovers and runs tests automatically. Your AI should show this structure, not just individual assert statements.

**Thinking Session Prompt**

What are edge cases and how do I identify them? When AI generates a function, what inputs should I always test? Give me a checklist I can use for any function.

### 18.2.3 Challenge It

**Thinking Session Prompt**

Here is a function and its AI-generated tests. Are the tests good enough?

```
def calculate_average(numbers):
    return sum(numbers) / len(numbers)

def test_average():
    assert calculate_average([1, 2, 3]) == 2.0
    assert calculate_average([10, 20]) == 15.0
```

**What to Look For**

The tests miss: empty list (ZeroDivisionError), single item list, negative numbers, very large numbers, non-numeric values. The function itself has a bug. It crashes on empty input. Good tests would catch this. AI-generated tests often test only the happy path.

### 18.2.4 What You Should Have Learned

- Good tests check edge cases, not just expected inputs
- pytest uses plain `assert`, simple and readable
- Test files: `tests/test_*.py`, functions: `test_*()`
- Arrange, Act, Assert. The test structure pattern
- Edge case checklist: empty input, None, wrong type, boundary values, single item
- Tests give confidence to modify code. That is their real value

## 18.3 The Gap

Testing is what separates code that works from code you can trust. When AI generates code, you cannot verify it by running it once with expected input. You need tests for the edge cases AI did not think of. Now you know how to write those tests.

In the Building Session, you will write tests for your chatbot's core functions.

## 18.4 Building Session

### 18.4.1 The Spec

Add pytest tests for your chatbot:

- Test `classify_input()` with greetings, questions, farewells, and unknown input
- Test `get_response()` returns the right type (tuple of string and bool)
- Test edge cases: empty input, very long input, input with only spaces
- Test `load_history()` when the file does not exist

### 18.4.2 Prompt It

**i** Building Session Prompt

Update my chatbot to v1.8. First, save your chatbot script as `chatbot.py` in your project folder so the tests can import it (Chapter 19 covers modules properly). Then create a test file `tests/test_chatbot.py` with pytest tests for:

- classify_input("hello") should return "greeting"
- classify_input("bye") should return "farewell"
- classify_input("what?") should return "question"
- classify_input("random text") should return "default"

- classify_input(“ “) should return”default” (not crash)
- get_response() should return a tuple of (str, bool)
- get_response(“bye”) should return (str, True). The exit flag

Use the arrange/act/assert pattern. Include at least one test that checks an edge case AI would miss.

### 18.4.3 Read the Code

Your AI will produce something like this:

```
"""Tests for the chatbot response system."""
from chatbot import classify_input, get_response


def test_classify_greeting():
    assert classify_input("hello") == "greeting"
    assert classify_input("hi there") ==
    "greeting"
    assert classify_input("Hey!") == "greeting"


def test_classify_farewell():
    assert classify_input("bye") == "farewell"
    assert classify_input("goodbye") == "farewell"


def test_classify_question():
    assert classify_input("what is Python?") ==
    "question"


def test_classify_default():
    assert classify_input("random text") ==
    "default"


def test_classify_empty():
    """Edge case: empty input should not crash."""
    assert classify_input("") == "default"


def test_classify_whitespace():
```

```
    """Edge case: whitespace-only input."""
    assert classify_input("   ") == "default"


def test_response_type():
    """get_response returns (str, bool)."""
    response, should_exit = get_response("hello")
    assert isinstance(response, str)
    assert isinstance(should_exit, bool)


def test_response_farewell_exits():
    """Farewell responses should signal exit."""
    _, should_exit = get_response("bye")
    assert should_exit is True


def test_response_greeting_continues():
    """Greeting responses should not exit."""
    _, should_exit = get_response("hello")
    assert should_exit is False
```

> **What to Notice**
>
> Each test function does one thing and has a descriptive name. Edge cases (empty input, whitespace) have their own tests with docstrings explaining why. `isinstance()` checks the type without caring about the specific value. The `_` in `_, should_exit` discards a value you do not need. Run with `pytest tests/` from the project root.

### 18.4.4 Stretch It

> **Building Session Prompt**
>
> Add a test for load_history() that uses pytest's tmp_path fixture to test with a temporary JSON file. Test both the case where the file exists with valid data and where it does not exist.

## 18.5 Your Chatbot So Far

- Ch 1-16: Full features with persistence and error handling

- Ch 17: Debug mode
- **Ch 18: pytest test suite for core functions**

## 18.6 Quick Reference

```
# Test file: tests/test_example.py
def test_addition():
    # Arrange
    x, y = 2, 3
    # Act
    result = x + y
    # Assert
    assert result == 5

# Run tests
# pytest                    # all tests
# pytest tests/test_x.py    # specific file
# pytest -v                 # verbose output

# Common assertions
assert result == expected
assert result is True
assert result is None
assert isinstance(result, str)
assert len(items) > 0

# Expected exceptions
import pytest
def test_invalid_input():
    with pytest.raises(ValueError):
        int("not a number")
```

# Chapter 19

# Modules and Packages

## 19.1 The Wall

AI wrote `import requests` at the top of your script. You ran it and got `ModuleNotFoundError: No module named 'requests'`. You had no idea whether `requests` was something you needed to install, something built into Python, or something AI made up entirely.

In another case, AI split your code into three files and used `from chatbot.responses import get_response`. You did not understand what the dots meant, where Python looked for files, or why it could not find the module you were staring at in your file browser.

This chapter fixes that.

## 19.2 Thinking Session

### 19.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> What is the difference between a module and a package in Python? When AI writes `import json` versus `import requests`, one works immediately and the other does not. Why? How do I know which modules come with Python (standard library) and which need to be installed?

Your AI should explain: the standard library (json, os, sys, datetime, random, pathlib) comes with Python. Third-party packages (requests, flask, pandas) need to be installed with `pip install`. A module is a single `.py` file. A package is a directory with an `__init__.py` file containing modules.

### 19.2.2 Go Deeper

**Thinking Session Prompt**

Explain the different ways to import in Python: `import module`, `from module import function`, `from module import *`, and `import module as alias`. When does AI use each one? Which are considered good practice and which should I avoid?

**What to Look For**

`import module` is safest. You always see where things come from (`module.function()`). `from module import function` is fine for specific, well-known names. `from module import *` is dangerous. It pollutes your namespace and you cannot tell where names come from. `import module as alias` is conventional for some libraries (`import numpy as np`).

**Thinking Session Prompt**

How do I organise my own code into multiple files? If I have a chatbot with separate files for responses, history, and the main loop, how do I import between them? What is `__init__.py` and why does AI put it in directories?

### 19.2.3 Challenge It

**Thinking Session Prompt**

What is wrong with these imports?

```
from os import *
import json
from my_project import helper  # file exists in
same directory
import pandas  # never installed
```

**What to Look For**

Line 1: `import *` imports everything from os, pollutes namespace with hundreds of names. Line 3: might fail depending on how you run the script, relative imports depend on the project structure and whether you are running as a script or module. Line 4: `ModuleNotFoundError` if pandas

is not installed. AI frequently imports packages it assumes are installed.

### 19.2.4 What You Should Have Learned

- Standard library vs third-party: `json` built-in, `requests` needs `pip install`
- `import module` is preferred over `from module import *`
- Packages are directories with `__init__.py`
- `pip install package_name` installs third-party packages
- `ModuleNotFoundError` means the module is not installed or not found
- Organise your own code: one concern per file, clear imports

## 19.3 The Gap

You can now read AI-generated import statements and know immediately whether something needs installation, comes with Python, or is a local file. When AI structures a project with multiple files, you understand how the pieces connect.

In the Building Session, you will split your chatbot into a proper package.

## 19.4 Building Session

### 19.4.1 The Spec

Split your chatbot into a package with multiple modules:

- `chatbot/`. The package directory
- `chatbot/__init__.py`, package marker
- `chatbot/config.py`, constants (BOT_NAME, VERSION, RESPONSES, KEYWORDS)
- `chatbot/responses.py`, classify_input() and get_response()
- `chatbot/history.py`, load_history() and save_history()
- `chatbot/main.py`. The main loop
- `run.py`, entry point that imports and runs main()

### 19.4.2 Prompt It

**i** Building Session Prompt

Restructure my chatbot into a package for v1.9. Create this structure: chatbot/ **init**.py (empty or with version) config.py (BOT_NAME, VERSION, RESPONSES dict, KEYWORDS dict) responses.py (classify_input, get_response, imports from config) history.py (load_history, save_history, imports

> json) main.py (main function, imports from other modules) run.py (entry point: from chatbot.main import main; main())
> Show me the import statements for each file and the key functions. Keep the existing features from v1.7.

### 19.4.3 Read the Code

Your AI will produce something like this:

```
# chatbot/config.py
BOT_NAME = "PyBot"
VERSION = "1.9"
HISTORY_FILE = "chat_history.json"

RESPONSES = {
    "greeting": ["Hello!", "Hi there!"],
    "farewell": ["Goodbye!", "See you!"],
    "question": ["Good question!", "Let me
    think..."],
    "default": ["Interesting!", "Tell me more."],
}

KEYWORDS = {
    "hello": "greeting",
    "hi": "greeting",
    "bye": "farewell",
    "goodbye": "farewell",
}
```

```
# chatbot/responses.py
import random
from chatbot.config import RESPONSES, KEYWORDS

def classify_input(text):
    words = text.lower().split()
    for word in words:
        if word in KEYWORDS:
            return KEYWORDS[word]
    if "?" in text:
        return "question"
    return "default"
```

```
def get_response(user_input):
    category = classify_input(user_input)
    responses = RESPONSES.get(category,
    RESPONSES["default"])
    return random.choice(responses), category ==
    "farewell"
```

```
# run.py
from chatbot.main import main

if __name__ == "__main__":
    main()
```

> **What to Notice**
>
> Each file imports only what it needs. `config.py` has no imports. It is pure data. `responses.py` imports from `config` using the package path `from chatbot.config import ...`. `run.py` is the entry point, one line. The `__init__.py` file can be empty; it just tells Python this directory is a package.

### 19.4.4 Stretch It

> **Building Session Prompt**
>
> Add a `chatbot/utils.py` module with a debug_log() function and a format_message() function. Import them in main.py. Also add `__version__ = "1.9"` to `__init__.py` so users can check `import chatbot; print(chatbot.__version__)`.

## 19.5 Your Chatbot So Far

- Ch 1-17: Full features with persistence, error handling, debug mode
- Ch 18: Test suite
- **Ch 19: Split into a proper package with separate modules**

## 19.6 Quick Reference

```
# Import styles
```

```
import json                             # full module
from datetime import datetime           # specific
item
import numpy as np                      # alias

# Install third-party
# pip install requests

# Package structure
# mypackage/
#     __init__.py
#     module1.py
#     module2.py

# Import from your package
from mypackage.module1 import my_function

# Common standard library
import json         # JSON data
import os           # file system
import sys          # system
import datetime     # dates/times
import random       # randomness
import pathlib      # file paths
```

# Chapter 20

# Objects

## 20.1 The Wall

AI used `class` and `self` everywhere and you had no idea why. It wrote `self.name`, `self.history`, `self.respond()`, every variable and function had `self` stuck on the front. You asked AI why and got a lecture about "object-oriented programming" and "encapsulation" that did not help you understand the actual code.

You tried changing `self.name` to just `name` and everything broke. You did not know why `self` was necessary or what it represented.

This chapter fixes that.

## 20.2 Thinking Session

### 20.2.1 Getting Oriented

> **Thinking Session Prompt**
>
> Explain Python classes and objects to me without jargon. I see AI using `class Chatbot:` and `self.name` constantly. What does `self` actually mean? Why can I not just use regular variables and functions? Give me a concrete example where a class is clearly better than functions.

Your AI should explain: a class bundles data (attributes) and behaviour (methods) together. `self` refers to the specific instance, if you create two chatbots, `self.name` is different for each one. Without classes, you would need separate variables for each chatbot's name, history, and settings. If your AI starts with abstract theory about "blueprints" and "objects," ask for a concrete example first.

### 20.2.2 Go Deeper

**Thinking Session Prompt**

Walk me through the parts of a Python class: `__init__`, methods, attributes, and the `self` parameter. Why does every method take `self` as the first parameter? What is `__init__` and when does it run? Show me with a simple example, not a theoretical one.

**What to Look For**

`__init__` is called automatically when you create an instance (`bot = Chatbot("PyBot")`). `self` is the instance being created or operated on. Python passes it automatically. Attributes set in `__init__` (like `self.name = name`) persist on the object. Methods are functions that operate on the object's data.

**Thinking Session Prompt**

What is inheritance in Python? AI sometimes writes `class EnhancedBot(Chatbot):`, what does the parentheses part mean? When would I use inheritance versus just adding methods to the original class?

### 20.2.3 Challenge It

**Thinking Session Prompt**

What is wrong with this class?

```
class Counter:
    count = 0

    def increment(self):
        count += 1

    def get_count():
        return count
```

What to Look For

Three bugs: (1) `increment` uses `count` instead of `self.count`. It tries to modify a local variable that does not exist. (2) `get_count` is missing `self` parameter. It cannot be called as a method. (3) `count = 0` at the class level is a class variable shared by all instances. It should be `self.count = 0` in `__init__` for per-instance counting.

### 20.2.4 What You Should Have Learned

- A class bundles data (attributes) and behaviour (methods)
- `self` refers to the specific instance
- `__init__` sets up the instance when created
- Every method takes `self` as its first parameter
- Inheritance lets you extend existing classes
- Use classes when you need multiple instances with their own state

## 20.3 The Gap

You now understand why AI uses classes. When you see `class Chatbot:` with `self.history` and `self.respond()`, you know that `self.history` belongs to that specific chatbot instance, and `respond()` operates on that instance's data. Classes are how AI structures any program that needs multiple instances of the same thing.

In the final Building Session, you will refactor your entire chatbot into a class.

## 20.4 Building Session

### 20.4.1 The Spec

Refactor your chatbot into a class:

- A `Chatbot` class with attributes for name, version, history, keywords, responses
- Methods: `greet()`, `classify()`, `respond()`, `save_history()`, `load_history()`
- An `__init__` method that sets up all state
- A `run()` method that contains the main loop
- The entry point creates an instance and calls `run()`

### 20.4.2 Prompt It

> **Building Session Prompt**
>
> Refactor my chatbot to v2.0 using a class. Create a Chatbot class that:
> - **init**(self, name, version) sets up: self.name, self.version, self.history (loaded from file), self.responses (the dictionary), self.keywords (the dictionary), self.message_count, self.debug_mode
> - greet(self), displays the welcome screen
> - get_user_name(self), asks for and validates the user's name
> - classify(self, text), classifies input using self.keywords
> - respond(self, user_input), returns (response, should_exit)
> - save_history(self), saves self.history to JSON
> - load_history(self), loads history from JSON
> - run(self). The main conversation loop
>
> Entry point: bot = Chatbot("PyBot", "2.0") bot.run()

### 20.4.3 Read the Code

Your AI will produce something like this:

```
"""PyBot v2.0: Object-oriented chatbot."""
import json
import random


class Chatbot:
    """A terminal chatbot with memory and
    pattern matching."""

    def __init__(self, name, version):
        """Initialise the chatbot."""
        self.name = name
        self.version = version
        self.message_count = 0
        self.debug_mode = False
        self.history = self.load_history()
        self.responses = {
            "greeting": ["Hello!", "Hi there!"],
            "farewell": ["Goodbye!", "See you!"],
            "question": ["Good question!"],
            "default": ["Interesting!", "Go
```

```
            on..."],
        }
        self.keywords = {
            "hello": "greeting",
            "hi": "greeting",
            "bye": "farewell",
        }

    def classify(self, text):
        """Classify input into a response
        category."""
        words = text.lower().split()
        for word in words:
            if word in self.keywords:
                return self.keywords[word]
        if "?" in text:
            return "question"
        return "default"

    def respond(self, user_input):
        """Generate a response."""
        category = self.classify(user_input)
        responses = self.responses.get(
            category, self.responses["default"]
        )
        return random.choice(responses), category
        == "farewell"

    def load_history(self):
        """Load conversation history from file."""
        try:
            with open("chat_history.json") as f:
                return json.load(f)
        except (FileNotFoundError,
        json.JSONDecodeError):
            return []

    def save_history(self):
        """Save conversation history to file."""
        with open("chat_history.json", "w") as f:
            json.dump(self.history, f, indent=2)
```

```
    def run(self):
        """Main conversation loop."""
        print(f"Hello! I'm {self.name}
        v{self.version}.")
        user_name = input(f"{self.name}: Your
        name? ").strip()
        print(f"Nice to meet you, {user_name}!")

        try:
            while True:
                user_input = input(f"{user_name}:
                ").strip()
                if not user_input:
                    continue

                self.message_count += 1
                response, should_exit =
                self.respond(user_input)
                print(f"{self.name}: {response}")

                self.history.append({
                    "user": user_input,
                    "bot": response,
                })

                if should_exit:
                    break
        except KeyboardInterrupt:
            print(f"\n{self.name}: Interrupted!")
        finally:
            self.save_history()


if __name__ == "__main__":
    bot = Chatbot("PyBot", "2.0")
    bot.run()
```

**What to Notice**

All state lives on `self`: `self.history`, `self.keywords`, `self.message_count`. Methods access this shared state through

`self`. You could create two chatbots with different names: `bot1 = Chatbot("PyBot", "2.0")` and `bot2 = Chatbot("Helper", "1.0")`. Each has its own state. The `if __name__ == "__main__"` guard means this file works both as a script and as an importable module.

### 20.4.4 Stretch It

**Building Session Prompt**

Create an EnhancedBot class that inherits from Chatbot and adds a mood system. Override the respond() method to adjust responses based on self.mood. Call super().respond() to get the base response, then modify it.

## 20.5 Your Chatbot So Far

- Ch 1-18: Full features split across functions and modules
- Ch 19: Package structure
- **Ch 20: Refactored into a Chatbot class with all features**

Your chatbot journey is complete. You started with 10 lines of print statements and built a full-featured conversational program with pattern matching, persistent memory, error handling, tests, and a clean class-based architecture.

You built it through conversation with AI, directing it, questioning it, and understanding every line it produced.

## 20.6 Quick Reference

```
# Define a class
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def speak(self):
        return f"{self.name} says Woof!"

# Create instances
dog1 = Dog("Rex", "Labrador")
dog2 = Dog("Bella", "Poodle")
print(dog1.speak())    # "Rex says Woof!"
```

```
# Inheritance
class Puppy(Dog):
    def __init__(self, name, breed, age_months):
        super().__init__(name, breed)
        self.age_months = age_months

    def speak(self):
        return f"{self.name} says Yap!"

# Special methods
class Item:
    def __str__(self):
        return "display string"
    def __repr__(self):
        return "debug string"
    def __len__(self):
        return 42
```

# Acknowledgments

This book grew out of teaching materials, handouts, exercises, lab worksheets, developed over years of introducing students to programming. Those materials were refined semester by semester as students showed me which explanations landed and which ones needed rethinking. The book is better for every student who said "I still don't get it" and forced a clearer explanation.

Colleagues who teach introductory programming provided the kind of honest feedback that improves books: what was missing, what was unnecessary, and where the difficulty curve was wrong.

This book is part of a series, and the ideas in it were sharpened against its siblings. *Think Python, Direct AI* pushed the conceptual foundations deeper. *Ship Python, Orchestrate AI* pulled the professional practices forward. Each book made the others better.

The Python community and the open source ecosystem behind Quarto, GitHub, and GitHub Pages made it possible to write, build, and publish without a traditional publisher. The tools are free, transparent, and remarkably good.

AI tools were used throughout the writing process. Claude (Anthropic) was a conversation partner for drafting, iterating, and refining. The process was exactly what the book advocates: consult the AI, evaluate its output, and make your own decisions. The author's judgement shaped every page. The AI made the work faster. It did not make the decisions.

# About the Author



**Michael Borck** is a software developer and educator passionate about the intersection of human expertise and artificial intelligence. He developed the Intentional Prompting methodology to help programmers maintain agency and deepen their understanding while using AI tools effectively.

Michael believes that the future of programming lies not in delegating to AI, but in conversing with it, treating AI as a collaborative partner that enhances human capability rather than replacing human understanding.

When not writing about AI collaboration, Michael works on practical applications of these principles across software development, education, and creative projects. He creates educational software and resources, and explores the 80/20 principle in learning and productivity.

---

**Connect**

- GitHub (https://github.com/michael-borck): code, books and projects
- books.borck.education (https://books.borck.education): the free book collection
- 8020workshop.com (https://8020workshop.com): passion projects and workshops
- LinkedIn (https://linkedin.com/in/michaelborck)

---

**Other Books in This Series**

*Foundational Methodology:*

- *Conversation, Not Delegation: Your Expertise + AI's Breadth = Amplified Thinking*
- *Converse Python, Partner AI: The Python Edition*

*Python Track:*

- *Think Python, Direct AI: Computational Thinking for Beginners*
- *Ship Python, Orchestrate AI: Professional Python in the AI Era*

*Web Track:*

- *Build Web, Guide AI: Business Web Development with AI*

*For Educators:*

- *Partner, Don't Police: AI in the Business Classroom*

# Appendix A

# Setting Up Python

This appendix covers how to install Python, choose an execution environment, and set up your development tools.

## A.1 Interpreted vs. Compiled Languages

Python is an *interpreted* language, your code is executed line by line rather than being compiled into machine code first. This makes Python excellent for learning and prototyping because you get immediate feedback.

## A.2 Ways to Run Python Code

### A.2.1 The Python Interpreter

The simplest way to run Python is the interactive interpreter. Open a terminal and type:

```
python3
```

You will see a prompt (>>>) where you can type Python statements and see results immediately:

```
>>> 2 + 2
4
>>> print("Hello!")
Hello!
>>> exit()
```

### A.2.2 IPython

IPython is an enhanced interactive shell with features like tab completion, syntax highlighting, and history search:

```
pip install ipython
ipython
```

Useful IPython features:

- **Tab completion**: type the start of a name and press Tab
- `?`: append to any object for documentation (`print?`)
- `%timeit`: measure execution time of a statement

### A.2.3 Python Scripts

For larger programs, save your code in `.py` files and run them:

```
python3 my_script.py
```

On macOS/Linux, you can make scripts directly executable:

```
#!/usr/bin/env python3
"""My first script."""
print("Hello from a script!")
```

```
chmod +x my_script.py
./my_script.py
```

### A.2.4 Jupyter Notebooks

Jupyter Notebooks combine code, output, and documentation in one file. They are ideal for learning, data analysis, and experimentation:

```
pip install jupyter
jupyter notebook
```

This opens a browser-based interface where you can create and run notebooks.

### A.2.5 Choosing the Right Environment

| Environment | Best For |
|---|---|
| Interpreter | Quick calculations, testing ideas |
| IPython | Interactive exploration, debugging |

| Environment | Best For |
|---|---|
| Scripts | Reusable programs, automation |
| Jupyter | Learning, data analysis, sharing |

## A.3 Installing Python

### A.3.1 Recommended: Miniconda

Miniconda provides Python plus the `conda` package manager in a lightweight install:

1. Download from docs.conda.io/en/latest/miniconda.html (https://docs.conda.io/en/latest/miniconda.html)
2. Run the installer and follow the prompts
3. Verify the installation:

```
python --version
conda --version
```

### A.3.2 Alternative: Official Python

Download from python.org (https://www.python.org/downloads/) if you prefer a minimal install without conda.

### A.3.3 Managing Packages with pip

pip is Python's standard package installer:

```
# Install a package
pip install requests

# Install a specific version
pip install pandas==2.0.0

# Install from a requirements file
pip install -r requirements.txt

# List installed packages
pip list
```

### A.3.4 Virtual Environments

Virtual environments isolate each project's dependencies:

```
# Create a virtual environment
python3 -m venv .venv

# Activate it
# macOS/Linux:
source .venv/bin/activate
# Windows:
.venv\Scripts\activate

# Install packages (isolated to this environment)
pip install pytest

# Deactivate when done
deactivate
```

> **Tip**
>
> Always use a virtual environment for each project. Add `.venv/` to your `.gitignore` file.

## A.4 Useful Libraries by Domain

| Domain | Libraries |
|---|---|
| Data Science | pandas, numpy, matplotlib |
| Web Development | Flask, Django, FastAPI |
| Automation | requests, beautifulsoup4, selenium |
| Testing | pytest, coverage |

Install what you need as you go, do not install everything at once.

## A.5 Setting Up an IDE

**VS Code** is recommended for beginners:

1. Install from code.visualstudio.com (https://code.visualstudio.com/)
2. Install the Python extension
3. Open your project folder
4. Select your Python interpreter (bottom status bar)

**PyCharm** is a full-featured alternative with built-in Python support.

**Jupyter Lab** extends Jupyter Notebooks with a more complete IDE-like interface:

```
pip install jupyterlab
jupyter lab
```

## A.6 Troubleshooting

**"command not found" errors:**

```
# macOS/Linux: add to your PATH
echo 'export PATH="$HOME/miniconda3/bin:$PATH"' \
    >> ~/.bashrc
source ~/.bashrc

# Windows (PowerShell as administrator):
$conda = "C:\Users\YourUsername\miniconda3"
$Env:Path = "$Env:Path;$conda;$conda\Scripts"
```

**Package conflicts:** Use virtual environments to isolate projects.

**Permission errors:**

```
# macOS/Linux
sudo chown -R $USER:$USER ~/miniconda3
```

**Mixed Python versions:** Use `python3` explicitly rather than `python` to ensure you are using the correct version.